# IMPLEMENTING A POST-QUANTUM SECURE ADAPTOR SIGNATURE SCHEME IN BLOCKCHAINS

A dissertation submitted to The University of Manchester
for the degree of Master of Science
in the Faculty of Science and Engineering

2026

Student id: 14202489

Department of Computer Science

# Contents

**Word Count: 9095**

# List of Tables

# List of Figures

# Abstract


Implementing a Post-Quantum Secure
Adaptor Signature Scheme in Blockchains
Student id: 14202489
A dissertation submitted to The University of Manchester
for the degree of Master of Science, 2026

Blockchains authenticate transactions with elliptic-curve signatures, which a large quantum computer would break. Basic post-quantum signatures are now standardised, but the *exotic* signatures that give blockchains their advanced features — in particular *adaptor signatures*, the cryptographic core of atomic swaps and payment channels — remain unevenly implemented in the post-quantum setting. This dissertation implements and evaluates one such scheme, LAS, a lattice-based adaptor signature, by reusing the CRYSTALS-Dilithium reference primitives, and demonstrates it end-to-end in a post-quantum atomic swap.

The artefact is built *additively*: the entire lattice core is reused unchanged, while the adaptor layer (PreSign, PreVerify, Adapt, Extract), a wire encoding, and the application are added as new code. It passes functional, serialization, tamper and known-answer tests, and an independent Rust implementation reproduces the wire format and pinned value byte-for-byte under a different compiler and harness.

The primary comparison is LAS against its own simplified Dilithium-style base at identical parameters and primitives, which isolates the adaptor layer's cost; a classical elliptic-curve (ECDSA) adaptor provides a functionality-matched "price of post-quantum" baseline. At the target core measurement boundary the adaptor layer adds small compute overheads: PreSign, PreVerify and Adapt add +2.2%,

+5.1% and +7.4% over the basic operations they mirror, and stay below +8.2% across the parameter sweep. The real cost is *communication* — the signature is 99.3% lattice response vector, and an adaptor swap additionally transmits a public-key-sized statement, which each adaptor operation must also decode, raising the computation premium to +15.8–84.4% once encoding is counted.

A further experiment tests whether the adaptor layer can coexist with ML-DSA's hint and rounding machinery. PreSign and PreVerify require adaptor-specific changes, but the standard FIPS 204 verifier accepts the adapted signature unchanged. This route roughly halves the signature at close to equal computation while leaving the statement untouched, relocating rather than removing the communication bottleneck.

Taking the protocol to a UTXO ledger, the setting the original construction assumes, sharpens the conclusion: measured across three configurations, the signature is at the application level not the dominant cost. The zero-knowledge proof of knowledge consumes 98.6–99.3% of each post-quantum swap, and substituting a lattice prover for a pairing-based one makes the swap *faster* while making it larger. Native on-chain verification also fits in one Ethereum transaction for the single setting and signature instance measured, and a Bitcoin client extended with one consensus rule settles a two-leg swap carrying no elliptic-curve signature. A working, tested, end-to-end post-quantum exotic signature is therefore achievable by reuse of standardised primitives, and the proof system, not the signature, is where a post-quantum swap should next be optimised.

# Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

# Copyright

# Statement of qualifications and research

The author holds a Bachelor of Informatics from Institut Sains dan Teknologi Terpadu Surabaya, Indonesia, awarded in 2024. As part of this degree, the author completed an undergraduate final project on Indoor People Counter Using Wireless 802.11 Channel State Information.

# Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Zhipeng Wang, for his guidance, support, and valuable feedback throughout this project. I am also deeply grateful to the Indonesia Endowment Fund for Education (LPDP) for its financial support and for providing me with the opportunity to pursue my Master's degree at the University of Manchester

# Chapter 1

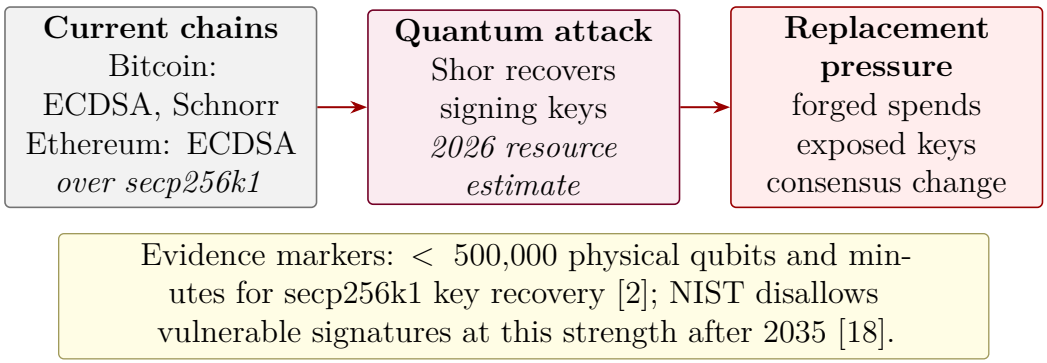
# Introduction

## 1.1 Context and motivation

Public blockchains commonly authorise transactions with the Elliptic Curve Digital Signature Algorithm (ECDSA) or Schnorr. Their security relies on the hardness of the elliptic-curve discrete logarithm problem, which Shor's algorithm solves in polynomial time on a sufficiently large quantum computer [23], allowing an adversary to forge signatures and spend other users' funds. A published 2026 resource estimate addresses that curve directly (fig. 1.1). A chain cannot treat that as distant: its record is permanent, so a break reaches keys already exposed, and changing how it verifies signatures requires a coordinated consensus-rule change, not merely a local software update. "Post-quantum" (PQ) cryptography replaces this assumption with problems believed hard even for quantum computers, such as those over lattices.

The U.S. National Institute of Standards and Technology (NIST) has standardised *basic* PQ signatures, notably ML-DSA (the Module-Lattice-Based Digital Signature Algorithm), derived from CRYSTALS-Dilithium [19, 9]. A basic signature authenticates a message and nothing more. Blockchains, however, depend on *exotic* signatures that add functionality on top of a basic scheme: multi-signatures, aggregate signatures, threshold signatures, and *adaptor signatures*. A recent survey documents that their exotic counterparts remain unevenly served by practical implementations in blockchain settings [6]. Turning one such paper design into a tested, measured artefact is the problem this project addresses.

**(a) Why current chain signatures need replacement**




**(b) Where LAS sits: basic vs. exotic × classical vs. post-quantum**

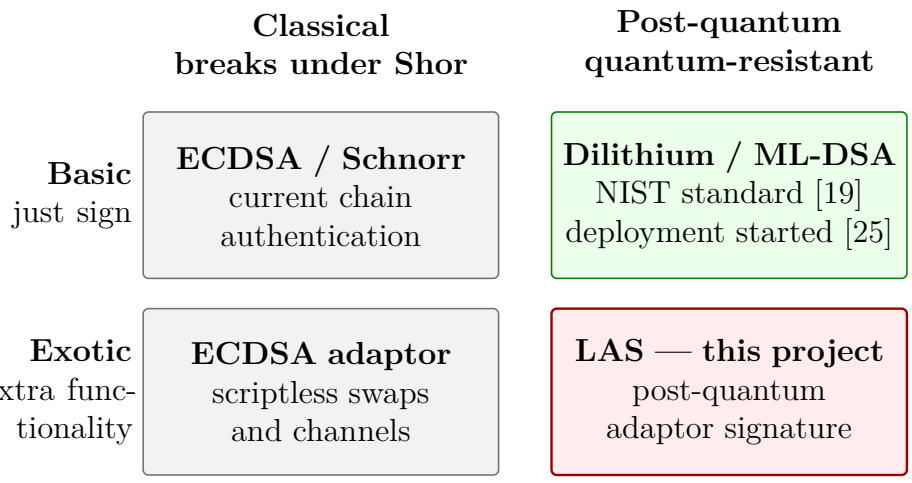



Figure 1.1: Why the problem is timely, and where this project sits within it. **(a)** The quantum argument is made from the signatures Bitcoin and Ethereum actually use, not from RSA: a published 2026 estimate gives the cost of recovering secp256k1 keys, while the 2035 marker is the independent NIST transition deadline for quantum-vulnerable signatures at this strength. **(b)** Basic post-quantum signatures are standardised, but blockchain applications need exotic signatures such as adaptors. That post-quantum exotic cell is the gap this dissertation addresses.

This dissertation focuses on the *adaptor signature*, which ties publishing a valid signature to simultaneously revealing a secret witness [20, 1]. That property makes adaptor signatures useful for scriptless blockchain protocols. In an atomic swap, two parties exchange assets across chains without a trusted intermediary

(fig. 1.2). They also apply to payment-channel networks, linking conditional off-chain updates through the same witness mechanism. The condition is embedded in the signature, so settlement verifies as an ordinary signature and needs no on-chain script for that condition [10, 6]. The exotic family is large, so the choice needs a reason. Multi-signatures and threshold signatures change who must cooperate to authorise a spend. Aggregate signatures compress many signatures into one. An adaptor signature serves a different purpose: it links two *separate* settlements. Once an adapted signature is published, a party holding the matching pre-signature can extract the secret and use it to complete the other settlement. That is the property an atomic swap needs, and the other three primitives do not provide it by themselves. Common classical adaptor constructions rely on discrete-logarithm assumptions, so they do not survive Shor's algorithm. Practical post-quantum alternatives remain limited. This distinct function, the applications that depend on it, and the implementation gap are why the adaptor signature was selected.

Cross-chain exchange first relied on custodial intermediaries; hash-time-locked contracts removed the custodian by locking both transfers under one hash preimage, so claiming one leg reveals the preimage unlocking the other. Such contracts need explicit on-chain script support, place a common hash on both chains — linking the legs — and enlarge the transactions. Moving the lock into the signature answers all three [20, 1]: settlements look like ordinary single-signer payments, atomicity is retained, and the witness reaches only the counterparty holding the matching pre-signature. The link removed is the explicit protocol one; timing and amounts can still correlate the legs.

## 1.2 Subject area and related work

**Lattice signatures and Dilithium.** CRYSTALS-Dilithium is a lattice signature in the "Fiat–Shamir with aborts" family [9, 17], its security reducing to the Module Learning With Errors and Module Short Integer Solution problems. It samples a masked commitment, derives a challenge by hashing, computes a response, and *rejects and retries* when the response would leak the secret — the rejection-sampling step characterising the family. Its reference implementation provides the polynomial arithmetic, number-theoretic transform (NTT) and SHAKE-based sampling this project reuses.

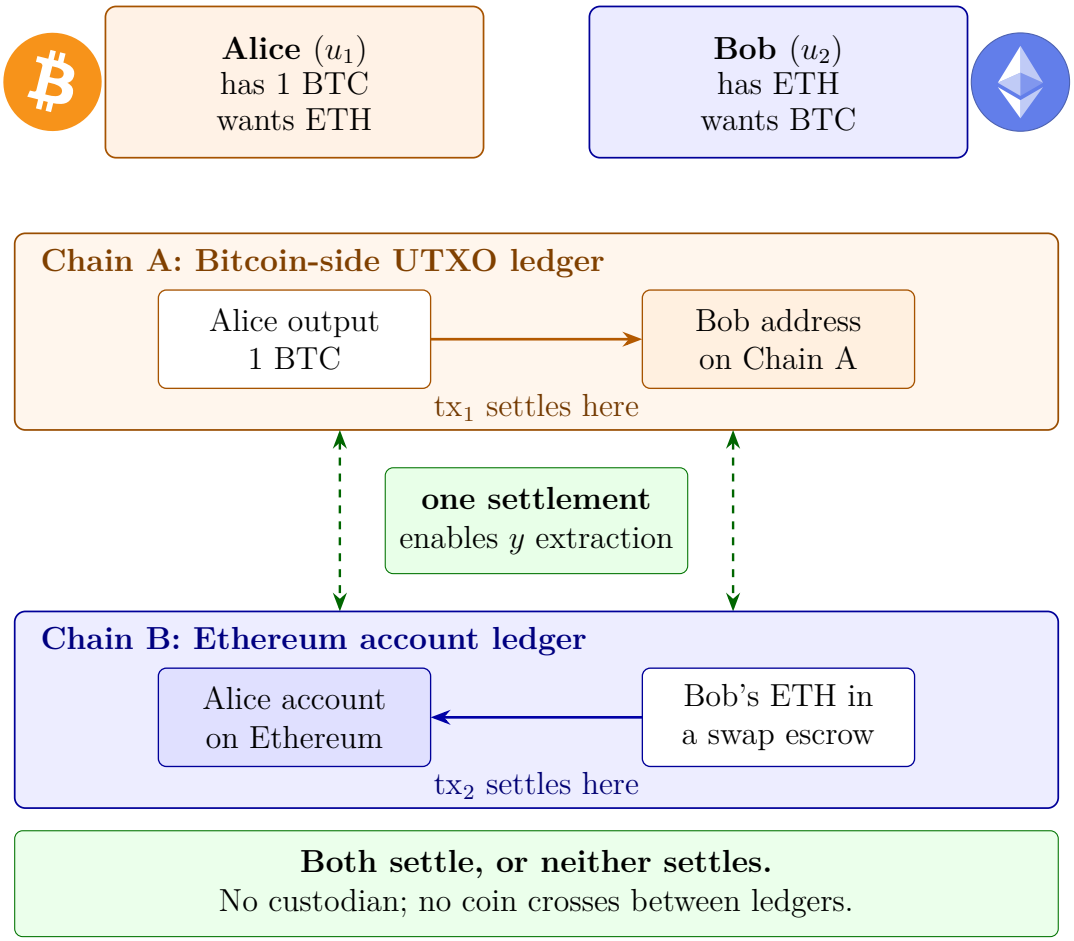



Figure 1.2: A motivation-level picture of an atomic swap, one concrete blockchain application of adaptor signatures. Alice and Bob trade assets recorded on two different ledgers: an unspent-transaction-output (UTXO) chain and an account-based one. Each payment settles on its own chain: $tx_1$ moves Alice's bitcoin to an address Bob owns, while $tx_2$ releases Bob's escrowed ether to an account Alice owns. The two venues place a lattice signature differently, which is why the Ethereum leg settles through a contract rather than as a plain payment (fig. 3.5). No coin crosses chains; the cross-chain object is the secret witness, which the counterparty extracts from the published signature using its matching pre-signature. Adaptor signatures provide this cryptographic link without requiring the swap condition to appear as a separate on-chain script. The two parties are named as in Esgin et al. [10], whose $u_1$ and $u_2$ the later chapters use; fig. 2.4 sets the exchange out message by message.

**Adaptor signatures.** Introduced informally as "scriptless scripts" [20] and later formalised [1], adaptor signatures add PreSign, PreVerify, Adapt and Extract to a base signature, relative to a hard relation $(Y, y)$.

**Post-quantum adaptor signatures.** Esgin et al. [10] proposed LAS, a lattice-based adaptor signature on a Dilithium-style base; it is the design implemented here. Their work is primarily theoretical — definitions, a construction and security proofs — leaving the practical, benchmarked, blockchain-facing implementation open. A second post-quantum route is isogeny-based: Tairi et al. [24] build IAS on CSI-FiSh, prove it secure in the quantum random oracle model, and report an implementation with measured costs. Kysil et al. [13] put basic PQ signature verification on Ethereum, ruling out Dilithium for its key sizes.

**Classical baseline.** The natural answer to "what does post-quantum cost" is a comparison with a *classical adaptor* signature, not merely plain ECDSA. Mature ECDSA-adaptor implementations exist [5]; this project benchmarks against one (chapter 2).

## 1.3 Aim and objectives

**Aim.** To implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a post-quantum blockchain atomic-swap application. The work is positioned as *systems implementation and evaluation*: it does not propose a new cryptographic protocol.

**Objectives.**

1. Review post-quantum and exotic-signature literature sufficiently to justify selecting the adaptor signature LAS and the Dilithium primitives.

2. Implement LAS additively on the Dilithium reference, reusing the lattice core unchanged and adding the adaptor operations and a byte-level wire encoding.

3. Validate correctness and robustness through functional, serialization, tamper, and known-answer tests, including cross-validation by a second, independent implementation checked against the same known-answer value.

4. Benchmark LAS fairly — at explicitly stated parameters, with repeated runs and reported variance — against a simplified Dilithium baseline at

identical parameters and a classical ECDSA-adaptor baseline, separating computation from communication cost across the five standard signature metrics.

5. Demonstrate an end-to-end post-quantum atomic swap using the implementation — following the protocol of [10], including its zero-knowledge proof of knowledge $\pi$ — and discuss its feasibility.

## 1.4 Contributions

This dissertation contributes a tested, independently cross-validated implementation of a post-quantum adaptor signature, and measurements of its cost. Four findings carry it. At the core tier, adaptor functionality measures close to free in computation over the simplified Dilithium-style base at identical parameters, but expensive in communication: bytes, not time (sections 3.2 and 3.4). The LAS paper's simplified Dilithium-style base omits ML-DSA's hint and rounding machinery; this project tested whether they could coexist with the adaptor layer. PreSign and PreVerify require adaptor-specific algorithms, yet an unmodified FIPS 204 verifier accepts the adapted signature (section 3.5). At application scale the bottleneck is not the signature but the zero-knowledge proof, leaving the fully post-quantum swap the faster of the two such configurations and the only one Shor does not break (section 3.7). Native on-chain verification, first measured far above the transaction gas cap, fits under it once the same predicate is re-expressed — under an unchecked registration invariant, at the one parameter set, message length and signature instance measured — while a settled swap sits well inside Bitcoin's standard-transaction weight limit (sections 3.8 and 3.9). All five objectives were met (table 4.1), subject to two limits throughout: no concrete security is formally analysed, and nothing ran on a live network.

## 1.5 Dissertation structure

Chapter 2 describes the methods; chapter 3 reports the measurements and discusses their validity; chapter 4 judges the work against the objectives; chapter 5 concludes, reflects critically, and proposes future work.

# Chapter 2

# Methodology

## 2.1 The LAS construction

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [10, 9]. All objects live in $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, the public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n \times (n+\ell)}$, and a key pair is a short secret $r$ with its image $t = A \cdot r$. The hard relation $(Y, r')$ reuses exactly that structure, so a statement is "just another public key": a sufficiently short preimage $r'$ of $Y$ under $A$ yields a Module-SIS solution $(r', -1)^\top$ for $[\, A \mid Y \,]$, hence the relation's hardness [10]. Esgin et al. [10] name this statement-generation step Gen: sample an independent ternary vector $r'$, compute $Y = A\,r'$, and return $(Y, r')$. Thus Gen is not a further primitive but KeyGen with its public key $t$ relabelled as the statement $Y$ and its secret $r$ relabelled as the witness $r'$. *Key generation is shared*, so LAS is exactly the base signature plus four adaptor operations. The implementation mirrors that split: `relation_gen` implements Gen, while `las.c` and `las.rs` hold the adaptor layer alone.

The base signature commits to a mask $y$ as $w = A\,y$, forms a challenge $c = H(pk, w, M)$, computes the response $\mathbf{z} = y + c\,r$ and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$. Relative to a statement $Y$, the adaptor extension [10] adds PreSign, which folds the statement into the hash, $c = H(pk, w + Y, M)$, keeps the response $\hat{\mathbf{z}} = y + c\,r$, and rejects at the *tighter* bound $\gamma - \kappa - 1$; PreVerify, which recomputes $w' = A\hat{\mathbf{z}} - c\,t$ and checks $c = H(pk, w' + Y, M)$; Adapt, which adds the witness $r'$, $\mathbf{z} \leftarrow \hat{\mathbf{z}} + r'$, producing a signature the base verifier accepts; and Extract, which recovers $s = \mathbf{z} - \hat{\mathbf{z}}$ (fig. 2.1).

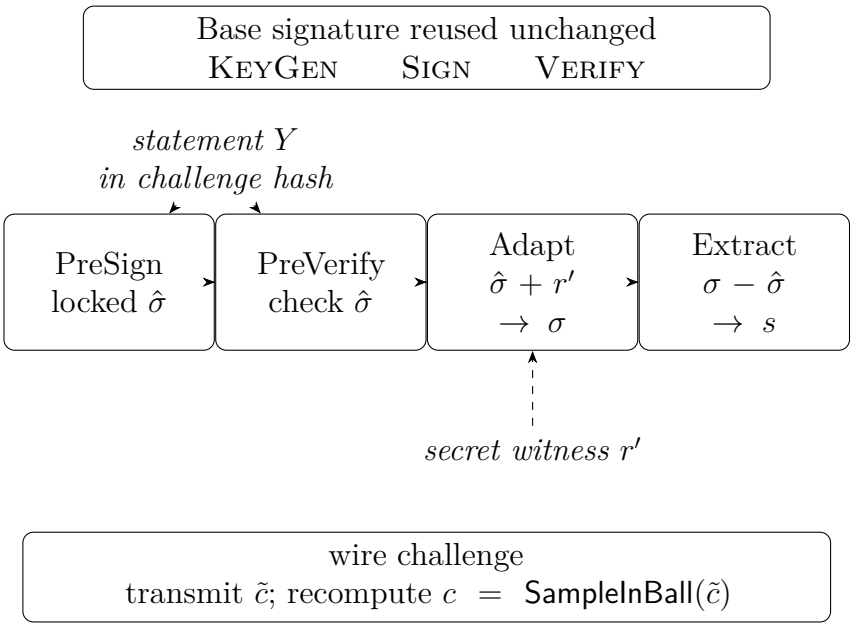



Figure 2.1: The four functions LAS adds around an unchanged base signature. PreSign and PreVerify bind the public statement $Y$ into the challenge; Adapt uses the witness $r'$ to turn the pre-signature into an ordinary signature; and Extract recovers a witness from that ordinary signature and the matching pre-signature. The implementation transmits $\tilde{c}$ and re-derives $c$ with SampleInBall.

$$\text{accept iff } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\, d\,(n + \ell). \tag{2.1}$$

The subtle design point is the bound budget. An *honest* witness is ternary, so $\|r'\|_\infty \le 1$, and because PreSign rejects at $\gamma - \kappa - 1$ the response after Adapt clears eq. (2.1); loosening it to $\gamma - \kappa$ would let adapted signatures exceed the bound and be rejected. Figure 2.2 shows the data flow.

## 2.2 Implementation strategy: reuse, do not reinvent

The guiding principle is that an exotic scheme equals a basic scheme plus extra functions, so the lattice arithmetic should be *reused*, not rewritten. The implementation is built additively on the upstream CRYSTALS-Dilithium reference [9, 7] at a pinned commit, and the same strategy is applied again in Rust (section 2.5). Neither is itself "the reference": both are *modified, simplified* ML-DSA-style bases

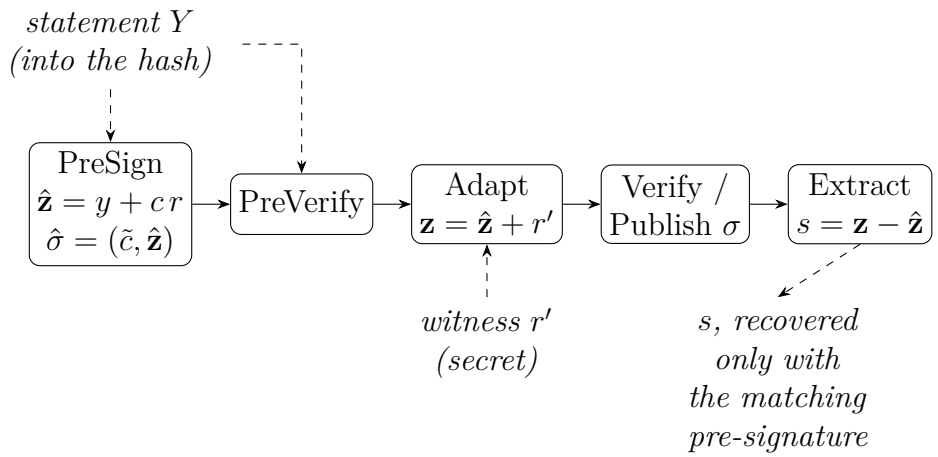



Figure 2.2: The LAS adaptor data flow, in the symbols of fig. 2.1: $y$ is PreSign's mask, $r'$ the honest witness, $s$ what Extract recovers. The statement $Y$ is folded into the challenge hash during PreSign and PreVerify; the witness is added by Adapt to turn the pre-signature into a base signature; and publishing that signature lets anyone holding the pre-signature run Extract to recover it. This last step is what an atomic swap uses.

carrying the LAS layer of [10].

Classifying every source file by how the reference is used (table A.8) yields a clean split: the entire lattice core is reused unchanged, while the adaptor scheme, serialization, application integration, and evaluation harnesses are project-specific additions. No upstream function is modified; the low-level Dilithium arithmetic and hash primitives are reused unchanged. The vendored `secp256k1-zkp` [5] and LaZer [15] are unmodified too, the latter behind a dedicated bridge.

**Data types and the matrix product.** LAS is a self-contained module over the reference's degree-$d$ polynomial type. Because $A = [\, I_n \mid A' \,]$, the product $A\,v = v_{\text{top}} + A' v_{\text{bot}}$ multiplies only $A'$.

**Hash, challenge, and samplers.** The random oracle $H$ is built from SHAKE256, absorbing a canonical encoding of the public key, then the commitment ($w$ for Sign, $w + Y$ for PreSign), then the message. The challenge sampler is the reference's `SampleInBall` at weight $\kappa$, giving $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary one for secrets and witnesses, and a rejection sampler uniform on $[-\gamma, \gamma]$ for the mask (appendix A.3).

**Simplifications, and the modulus.** Following [10], the base signature omits Dilithium's hint vector, Power2Round and high/low-bit decomposition, hashing the full commitment $w$ rather than its high bits. This keeps the exact identity $A\mathbf{z} - c\,t = w$ available, at the cost of larger keys and signatures. Whether those optimisations can coexist with an adaptor layer was not assumed but *tested* (section 3.5). The modulus is reused too: the reference NTT fixes $q = 8\,380\,417 \approx 2^{23}$, FIPS 204's own modulus [19], rather than the $q \approx 2^{24}$ of [10] — correctness is unaffected since $q > 2\gamma$, and only the Module-SIS/LWE margin differs. No module depends on a mode-specific constant, so identical code runs at every parameter set.

## 2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs, so a bit-packed wire encoding, a typed codec and a verify-from-bytes entry point were implemented. Two decisions matter. Validation is *asymmetric*: keys and witnesses are checked on decode, a signature is not, so a tampered response is caught by Verify — the entry point the tamper test attacks. And, following FIPS 204 [19], a signature carries not the challenge polynomial $c$ but the digest $\tilde{c}$ from which verification re-derives it, under the standard's $\lambda/4$ rule (appendix A.5).

## 2.4 Testing methodology

Four test types cover progressively weaker assumptions about the environment, stated in full in table A.2. Functional tests assume an honest caller and assert the whole adaptor cycle, including the *tripwire* clause — a pre-signature must *fail* basic verification — which is what distinguishes an adaptor signature from a signature. Serialization and tamper tests assume a hostile caller; the known-answer test assumes a different machine, and its pinned value is what the second implementation is checked against (section 2.5).

## 2.5 An independent second implementation: the Rust port

The complete scheme was implemented a second time in Rust on the same reuse principle, layered on a fixed version of the `fips204` crate [22], with the LAS layer added as new modules. The wire format is byte-identical to the C encoder's by construction (fig. 2.3).

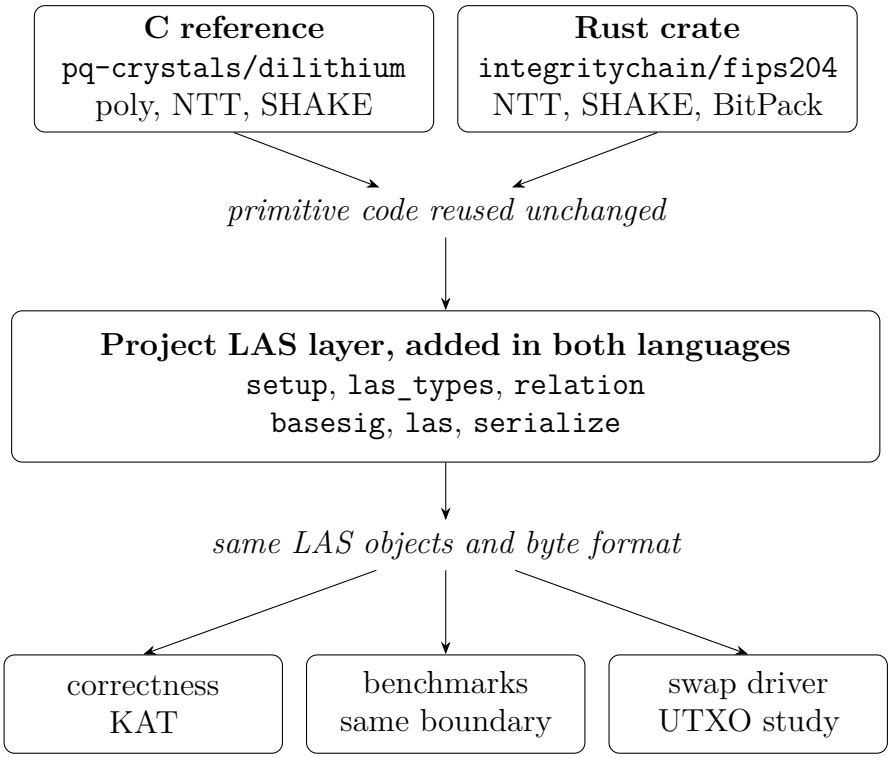



Figure 2.3: Implementation structure and C/Rust correspondence across the two implementations. C uses the vendored `pq-crystals/dilithium` reference [7], Rust uses the `integritychain/fips204` crate [22], and neither modifies upstream primitive functions. The project LAS layer is added in both languages with the same module split and packed object format; tests, known-answer vectors, matched benchmarks, and the UTXO swap driver are produced from that layer. The per-file classification is table A.8.

The port serves three purposes. *Cross-validation*: short of a formal proof, the strongest practical argument for correctness is a second implementation agreeing on pinned vectors. *An independent benchmark methodology*: Criterion.rs [12] shares no timing code with the C harness, so agreement defends the C numbers against harness artefacts. *Deployment realism*: a memory-safe language is the

plausible production vehicle.

To make the timings comparable the two protocol drivers mirror each other exactly — same repetition scheme, fixed seed, fixed message, success-path gating and rejection statistics. One caveat qualifies the port's independence: Rust follows the C implementation's early-abort norm check, an implementation detail not prescribed by the LAS construction [10]. Sharing that is what makes the two languages time the same algorithm, but it is established by inspecting the two norm checks rather than by the automated evidence, which constrains outputs and rejection rates, not the work profile of a rejected attempt (appendix A.3).

## 2.6 Benchmarking methodology and fair comparison

The evaluation separates *computation* cost (wall-clock time per operation) from *communication* cost (bytes transmitted or stored).

**Measurement boundaries.** Undocumented boundaries are a recognised benchmarking pitfall for lattice signatures [21], so every timing belongs to one of the three declared boundaries of table A.4: a *core tier* isolating the adaptor algebra from encoding, a *packed tier* giving what an on-chain consumer pays, and, for the classical baseline alone, the *hybrid* boundary its library exposes. Comparisons stay within one boundary; the primary base-versus-LAS comparison shows *both*.

**Implementation of record, repeatability, and machine.** Unless a caption says otherwise, timings are the C implementation's, with the Rust results reported separately and never averaged with them (table 3.3). Driver timings are a mean ± sample standard deviation over each caption's repetitions; Criterion reports bootstrap 95% confidence intervals. All non-random inputs are fixed, so variability comes only from rejection sampling and machine noise, never from input selection [21]. Toolchain, hardware and per-table commands: appendix A.1.

**Per-operation reporting, not cumulative.** Every operation is timed independently: they are split across parties, so a combined figure would average over costs never paid together and hide the per-operation overhead.

**The primary comparison: LAS against its own simplified base.** LAS is exactly its base signature plus four operations at identical algorithm, parameters and primitives, so each base-versus-LAS comparison holds all three fixed and varies only the adaptor layer (justified in section 4.1). Each adaptor operation is paired with the base operation it mirrors — ADAPT against VERIFY, since it runs a PreVerify before adding the witness — and EXTRACT, having no base analogue, is reported separately.

**Rejection sampling: the expected attempt count.** Sign-class operations restart until the response passes its norm bound, so no timing claim is meaningful without the attempt statistics. Sign accepts each coefficient with probability $\big(2(\gamma-\kappa)+1\big)/(2\gamma+1)$ *independently of the secret* — why the response is simulatable (appendix A.3) — and an attempt succeeds only if all $(n+\ell)\,d$ pass:

$$p_{\text{acc}} = \left(\frac{2(\gamma-\kappa)+1}{2\gamma+1}\right)^{(n+\ell)d} \approx \left(1-\frac{\kappa}{\gamma}\right)^{(n+\ell)d} \approx e^{-\kappa(n+\ell)d/\gamma} = e^{-1} \approx 36.8\%, \tag{2.2}$$

the last step using the design choice $\gamma = \kappa\, d\,(n+\ell)$ from [10], made exactly so the expected number of restarts is "about $e < 3$". Expected attempts are $1/p_{\text{acc}}$, and PreSign's bound being tighter by one, it is *expected* to restart slightly more often before any adaptor arithmetic is counted.

**Rejection sampling: the run-validity gate.** Because the attempt count is geometric, per-call timings are heavy-tailed and averages over too few calls can drift several percent on rejection luck alone [21]. The drivers therefore treat the attempt statistics as a *validity gate*: measured mean attempts must match the closed-form expectation within five standard errors, and a failing run aborts instead of producing evidence. This is a consistency check on the rejection rate, not a proof of sampler correctness (appendices A.2 and A.3). A second gate precedes every timing: the correctness contract is asserted on the objects about to be timed, so no failure path is ever measured.

**Parameter sensitivity and component sizes.** To test the adaptor overhead's sensitivity to the parameter set, the same comparison is repeated at the four parameter sets of table 2.1 (symbols in table A.3). Each key and signature is decomposed into components, so the *cause* of a size difference can be identified,

not merely its total stated.

Table 2.1: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set is *paper-derived* rather than that paper's exact set: it takes the dimensions of [10], a historical reference point corresponding to no ML-DSA set, but runs at FIPS 204's $q \approx 2^{23}$ [19] rather than the $q \approx 2^{24}$ of [10]: correctness is unaffected ($q > 2\gamma$), while the security margin changes. The *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5, aligning the reusable ML-DSA primitives and the challenge-digest strength $|\tilde{c}|$ with ML-DSA-44/65/87 while retaining LAS's own ternary secret distribution, rejection bounds, exact hint-free relation and unoptimised serialization. They are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims: a matched dimension is not a claim that LAS *is* the corresponding ML-DSA parameter set or inherits its security category. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table A.3 defines each symbol.

| Setting | $n$ | $\ell$ | $M$ | $\kappa$ | $\gamma$ | $d$ | $q$ |
|---|---|---|---|---|---|---|---|
| LAS-2020/845 reference | 4 | 4 | 8 | 60 | 122 880 | 256 | 8 380 417 |
| Simplified Dilithium-II | 4 | 4 | 8 | 39 | 79 872 | 256 | 8 380 417 |
| Simplified Dilithium-III (target) | 6 | 5 | 11 | 49 | 137 984 | 256 | 8 380 417 |
| Simplified Dilithium-V | 8 | 7 | 15 | 60 | 230 400 | 256 | 8 380 417 |
| Classical (secp256k1 ECDSA adaptor) | ≈128-bit classical (see table 3.5) | | | | | | |

**The classical baseline: functionality-matched, not security-matched.** The second baseline is the `secp256k1-zkp` library's `ecdsa_adaptor` module [5], implementing Fournier's construction [11], unmodified at a pinned commit. It is a functionality-matched "price of post-quantum" baseline, not a same-security one; plain ECDSA is excluded, the fair counterpart of an adaptor signature being another adaptor signature.

## 2.7 Application methodology: the UTXO swap study

This section defines the application protocol and the measurement boundary used for the Stage-2 results. The protocol follows Figure 1 in [10] over two UTXO ledgers; section 3.6 later reports the execution evidence, while sections 3.7 and 3.8 report cost.

**The venue, and why it is modelled.** The construction [10] states its applications over "a UTXO-based blockchain like Bitcoin *where the signature algorithm is replaced with a lattice-based signature scheme*". That sentence is implemented literally: a UTXO ledger was built in which *the signature algorithm is a parameter*, so one ledger serves every configuration and any measured difference is the cryptography, not the ledger. It is a model, not a node; the boundary is set out in appendix A.4.

**The protocol that is executed.** Figure 2.4 is the run book implemented by the driver. Alice, written $u_1$ in the paper notation, holds the witness $y$ and therefore moves first; Bob, written $u_2$, only pre-signs after Alice's pre-signature and, in the LAS configurations, her proof $\pi$ have been verified. That proof is part of the method, not decoration: it proves the statement has a short witness, closing the knowledge gap that would otherwise let a malicious first party leave an unadaptable pre-signature behind. No UTXO is spent while the pre-signatures are exchanged: a settlement occurs only when a transaction carrying an adapted signature is broadcast to its own ledger. The other ledger never verifies a foreign transaction; atomicity comes from Bob observing $\sigma_2$ on ledger B, extracting the witness, and using it to finish the spend on ledger A. The signed message $m_i$ is the template $tx_i$ serialized without its input signatures, not the settled transaction bytes.

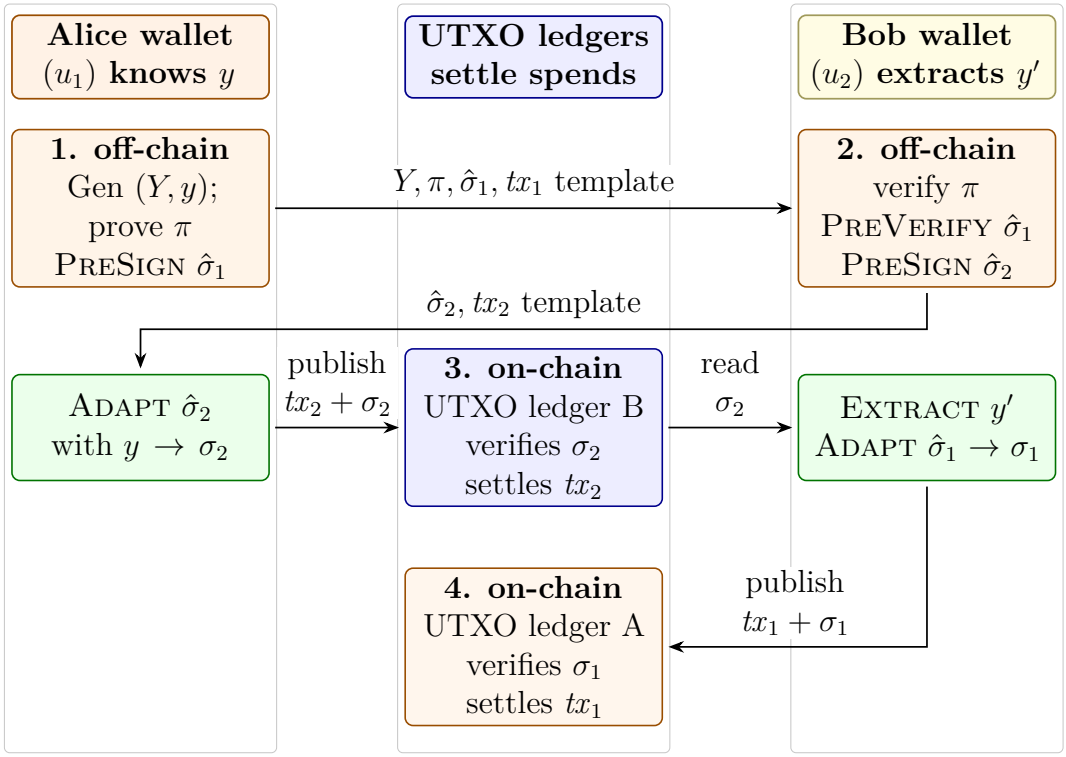



Figure 2.4: The atomic-swap protocol used in the UTXO study, adapted from Figure 1 in [10]. Protocol messages are exchanged off-chain; only the completed spend transactions are submitted to the two UTXO ledgers. The secret witness is never transmitted.

**What a settled transaction contains, and what the adaptor layer changes.** Of the adaptor-protocol objects exchanged by the swap, only the adapted signature $\sigma$ reaches a ledger, occupying the slot in which an ordinary signature already appears — since SegWit [16], a segregated *witness* field. The statement $Y$, the proof $\pi$ and both pre-signatures stay off-chain, while the witness $y$ is never transmitted: $u_2$ derives it from the published $\sigma$ and its own pre-signature. The construction therefore adds *no adaptor-specific on-chain field* — what *scriptless* means: the swap condition need not appear as a separate on-chain script. The resulting transaction cost is not modelled but read from two *mined* regtest transactions: the client's own size, weight and virtual size, with the field decomposition reconstructed from the decoded transactions and checked against the weight accounting of [16, 14, 26] (section 3.8).

One notational point matters once the venue is a real ledger. The swap protocol writes the signed object as $tx_i$, but a transaction is not a message: a signature commits to its *signature hash*. Three objects are therefore kept distinct:

the unsigned template exchanged off-chain, the message derived from it that PreSign receives, and the settled transaction. Conflating them would misreport communication cost.

**Three configurations, and what is controlled.** The three configurations of table 2.2 each run the swap protocol end-to-end across two ledgers. The *role-A* proof is the proof of knowledge $\pi$ the protocol places on the wire immediately after $Y$, claiming a witness to $Y$ exists and is short. Only the $2 \to 3$ step is controlled: both LAS configurations share one expansion of the public matrix, derive all key material from one pinned seed, and prove the *identical* relation $\exists\, r' : Ar' = t' \wedge \|r'\|_\infty \leq 1$ — the hard relation of the statement/witness pair, *not* the signer's key pair, and including the norm bound, since proving only $Ar' = t'$ would fail to close the knowledge gap. This is verified rather than assumed: a configuration whose two halves disagree on a *relation binding* each computes independently refuses to run (appendix A.3). The $1 \to 2$ step is not controlled — it changes the signature scheme, the relation and whether a role-A proof exists — so it is a whole-stack comparison, the signature-only question being answered by section 3.4.1.

Table 2.2: The three configurations. Only the $2 \to 3$ step is controlled, so those two differ in the proof system alone; configuration 1 differs in signature scheme, relation and proof alike. Its missing role-A proof is a finding rather than an omission: an ECDSA adaptor has no knowledge gap, since extraction recovers the discrete logarithm exactly, so the unmodified `secp256k1-zkp` construction needs no $\pi$ — and adding one would have measured a modified baseline rather than the library as it stands. The DLEQ proof carried *inside* its pre-signature is a different object and is not counted as $\pi$: its author is the pre-signer, and its claim is pre-signature well-formedness, not knowledge of the role-A statement witness $r'$. Its cost is reported in the bytes by which a pre-signature exceeds a signature.

| # | Signature | Role-A proof $\pi$ | Setup | Fully PQ |
|---|---|---|---|---|
| 1 | ECDSA adaptor signature (libsecp256k1-zkp) | none required | — | no |
| 2 | LAS (simplified Dilithium-III) | Groth16 over BN254 (pairing-based zk-SNARK) | trusted | no |
| 3 | LAS (simplified Dilithium-III) | LaZer (LNP22 linear relation with norms) | transparent | yes |

**The Groth16 circuit.** No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and cheaper than the statement suggests: $r' \mapsto Ar'$ is *linear with public coefficients*, needing no witness–witness multiplication, the expensive ingredient in a rank-one constraint system (appendix A.3).

**Measurement.** Because gas does not exist on this venue, the axes are execution time and communication cost, the latter counting *off-chain* protocol messages, not only what reaches a ledger. Sizes are observed from the transmitted buffers rather than computed, and these timings sit at the packed tier, so they are not comparable with core-tier figures elsewhere. One-time costs are excluded and reported separately (appendix A.3).

# Chapter 3

# Results and discussion

This chapter reports what was measured and why the results should be believed: correctness, computation cost, communication cost, and the application results.

## 3.1 Correctness and robustness

No performance claim means anything until the implementation is shown *correct*, and an adaptor signature is not correct merely because it signs and verifies — it must satisfy a specific contract. Table 3.1 lists every clause; all pass. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire), becomes an ordinary signature once the witness is added, and completing it reveals that witness — the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness, clause 7 reproducibility.

## 3.2 Computation cost

The primary computation result is the cost of the *adaptor layer*. The two paths share algorithm, parameters and primitives, so any difference is purely the price of adaptor functionality (section 2.6). Table 3.2 reports it at *both* boundaries and fig. 3.1 visualises the core tier, where every paired adaptor operation at the target setting stays within single digits of its counterpart, Extract is the cheapest of all, and every operation completes in well under two milliseconds.

At the core boundary the adaptor layer is almost free, and the reasons are structural. PreSign (+2.2% over Sign) runs the same reject loop, adding only $Y$ to the commitment and a tighter bound; PreVerify (+5.1%) recomputes the same

Table 3.1: The adaptor-signature correctness contract at the target setting, with the test that checks each clause in the evidence run of record; all pass. Clauses 1–4 and 5a are asserted on every one of the 1000 randomised iterations of the functional suite `test_las`; 5b and 6 by `test_serde`, which also performs 256 randomised serialization round trips; 7 by the known-answer test `test_kat`, whose pinned cross-language value binds the packed public key, secret key, signature, pre-signature and adapted signature over four fixed vectors. PreVerify and Extract are asserted in all three tests, but produce no object that value covers.

| # | Property | Result |
|---|---|---|
| 1 | PreSign produces a pre-signature that PreVerify accepts | pass |
| 2 | the pre-signature *fails* basic Verify (statement-binding tripwire) | pass |
| 3 | Adapt yields a signature that basic Verify accepts | pass |
| 4 | Extract recovers the witness *exactly* ($y' = y$) | pass |
| 5a | a tampered message fails Verify | pass |
| 5b | each of the 6736 low-bit flips of a packed-signature byte is rejected | pass |
| 6 | malformed bytes rejected on decode ($pk \geq Q$, ternary code-3), out-of-range objects on encode ($z$, non-ternary secret) | pass |
| 7 | the deterministic API is byte-identical on re-run | pass |

commitment shape with the extra $+Y$; Adapt (+7.4%) runs a PreVerify then adds the witness vector; Extract merely subtracts $s = z - \hat{z}$ and checks $A\,s = Y$. None changes the asymptotic work. This is the headline of the standalone-signature evaluation: *at the core tier* adaptor functionality is single-digit at the target setting and stays below +8.2% across the parameter sweep. The supporting sensitivity plot is deferred to Appendix A.9.1, where fig. A.1 records the same comparison across all four parameter sets. Adapt and Extract cost about a verification or less each (table 3.2).

At the packed boundary the premium grows to +15.8%, +37.9% and +84.4% — several times the core-tier percentages. The increase is codec work on the adaptor's own objects rather than adaptor arithmetic: PreSign and PreVerify additionally decode the statement, while Adapt also decodes the witness and re-encodes the signature. A deployment working from bytes should budget for the packed row — reported lattice timings are notoriously sensitive to this boundary [21].

Signing and pre-signing take longer than verification because of rejection sampling, intrinsic to the Fiat–Shamir-with-aborts family rather than a defect here. Equation (2.2) predicts 2.719 attempts per Sign and 2.775 per PreSign, the latter restarting about 2% more often because its bound is tighter by one;

Table 3.2: The primary computation result: per-operation adaptor overhead at the target setting *Simplified Dilithium-III* (table 2.1), at *both* measurement boundaries of section 2.6. Each LAS adaptor operation is paired with the basic operation it mirrors; "Overhead" is the extra cost as a percentage of that basic operation, within the same tier. Timings are $\mu$s per operation, measured on the C implementation, the implementation of record (mean ± sample standard deviation over 5 repetitions × 500/1000 iterations; machine of record in section 2.6); the Rust port's independent measurements are in table 3.3. KeyGen, Sign, and Verify are reused unchanged by LAS, and Gen executes exactly the same computation as KeyGen, so the measured KeyGen timing also gives the cost of relation generation. Extract has no basic analogue. The packed-tier overheads are larger because the byte API does codec work on the adaptor's own objects that the basic operation does not — serialization rather than adaptor arithmetic, itemised per operation in the text; the core tier isolates the adaptor computation itself. Absolute core-tier timings at every parameter set are in table A.6 (appendix).

| Operation (basic / adaptor) | Basic ($\mu$s) | LAS adaptor ($\mu$s) | Overhead |
|---|---|---|---|
| *Core tier (structures in/out — isolates the adaptor arithmetic)* | | | |
| KeyGen | $44.4 \pm 1.4$ | $44.4 \pm 1.4$ (shared) | — |
| Sign / PreSign | $403 \pm 17$ | $412 \pm 10$ | +2.2% |
| Verify / PreVerify | $91.6 \pm 0.5$ | $96.2 \pm 1.6$ | +5.1% |
| Verify / Adapt | $91.6 \pm 0.5$ | $98.3 \pm 0.9$ | +7.4% |
| Extract | — | $35.9 \pm 0.5$ | (LAS only) |
| *Packed tier (wire bytes in/out — incl. decode + encode)* | | | |
| KeyGen | $146 \pm 0.21$ | $146 \pm 0.21$ (shared) | — |
| Sign / PreSign | $642 \pm 36$ | $743 \pm 9$ | +15.8% |
| Verify / PreVerify | $320 \pm 15$ | $441 \pm 3$ | +37.9% |
| Verify / Adapt | $320 \pm 15$ | $589 \pm 9$ | +84.4% |
| Extract | — | $445 \pm 3$ | (LAS only) |

counted directly rather than estimated from a timing ratio, the run of record observed 2.696 and 2.804. The supporting rejection table and CDF are deferred to Appendix A.9.1: table A.5 and fig. A.2 show that measured rejection behaviour matches the geometric model and that every measured row passes the five-standard-error validity gate of section 2.6. At 2500 timed calls the gate band ($\pm\sim 8\%$) is wider than that 2% separation, so the core-tier PreSign overhead (+2.2%) should be read as of the same order as the prediction rather than as a precise constant; only the $\approx 10^5$-call Criterion harness resolves it.

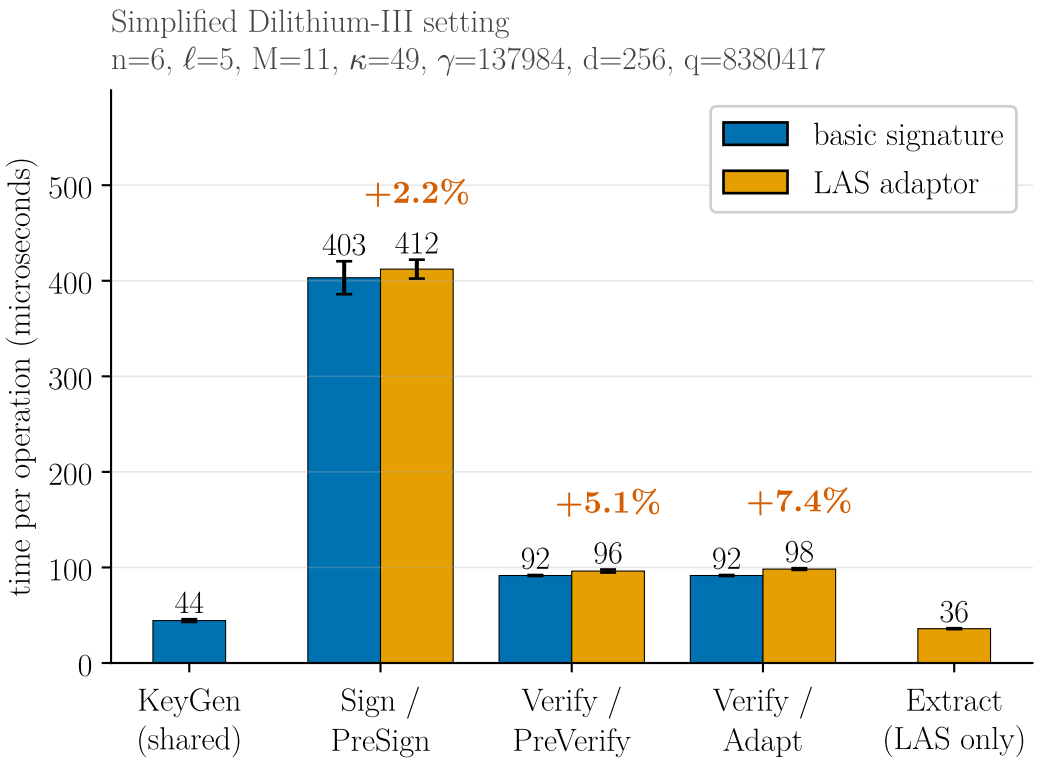



Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot), measured on the C implementation (the implementation of record, section 2.6). The percentage above each orange bar is the adaptor overhead over its blue partner. KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. Error bars are the sample standard deviation. Exact means, the repetition counts and the packed tier are in table 3.2; the same comparison across all four parameter sets is the parameter-sensitivity check in fig. A.1.

## 3.3 Cross-implementation check: C implementation versus Rust port

The scheme exists twice (section 2.5), and at the byte level the two implementations agree exactly: the same packed sizes (public key 4416 B, signature 6736 B,

checked programmatically on every Rust run) and the same pinned known-answer value. Since that value binds the packed key pair, signature, pre-signature and adapted signature, a divergence escapes it only by leaving every one of those bytes unchanged.

Table 3.3: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* ($\mu$s per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions $\times$ 500/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

| Operation | C implementation ($\mu$s) | Rust port ($\mu$s) | Rust / C | Rust, Criterion ($\mu$s, 95% CI) |
|---|---|---|---|---|
| KeyGen | $44.4 \pm 1.4$ | $131 \pm 12$ | 2.96 | 102 [101, 102] |
| Sign | $403 \pm 17$ | $1087 \pm 35$ | 2.70 | 926 [921, 931] |
| Verify | $91.6 \pm 0.5$ | $239 \pm 6$ | 2.61 | 190 [190, 191] |
| PreSign | $412 \pm 10$ | $1125 \pm 21$ | 2.73 | 959 [953, 965] |
| PreVerify | $96.2 \pm 1.6$ | $237 \pm 6$ | 2.46 | 190 [189, 190] |
| Adapt | $98.3 \pm 0.9$ | $249 \pm 14$ | 2.53 | 191 [191, 192] |
| Extract | $35.9 \pm 0.5$ | $104 \pm 19$ | 2.89 | 70.0 [69.9, 70.1] |

Absolute timings do not agree: every operation is slower in the Rust port, by up to 196% (KeyGen), and no cause for that gap has been measured. What reproduces is the comparison itself. At the core tier the Rust port measures +3.5%, $-0.7\%$ and +4.2% — all single-digit, like the C column beside them; the slightly *negative* PreVerify figure is the measurement floor rather than a real speed-up, the genuine overhead at this scale being smaller than the gap between the two verify paths. The packed tier reproduces too (+1.4%, +17.5%, +30.9%), all positive and in the C figures' order. Both ports pass the same rejection gate (fig. 3.2). The relative adaptor overheads, not the absolute timings, are therefore properties of the algorithm rather than of one codebase, compiler or harness.

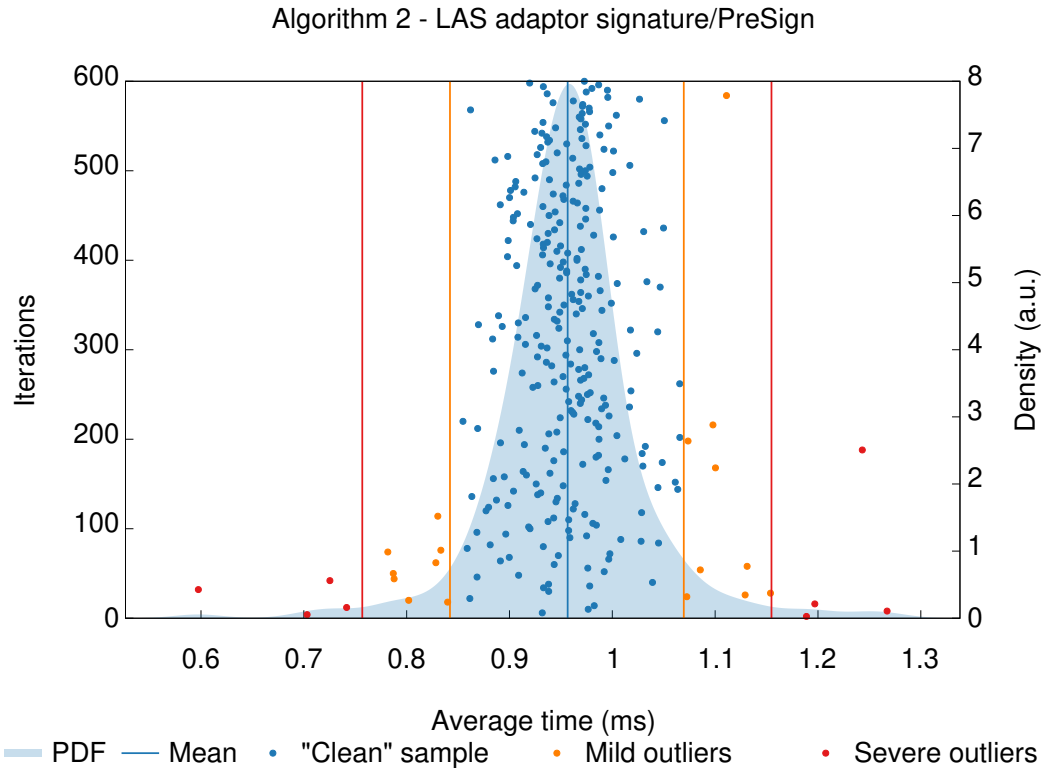



Figure 3.2: The statistical harness's own report for PreSign at the target setting, reproduced from Criterion.rs: 300 timed samples (dots), the kernel-density estimate of the sampling distribution (shaded), the mean (vertical blue line), and Criterion's automatic outlier classification (mild / severe, orange / red). Three presentational changes were made to Criterion's SVG: the type enlarged, the legend moved from the right-hand column into the row beneath the plot, and the margins re-flowed and cropped. Inside the plot frame the coordinate map is the identity, so no plotted value, curve, mean, axis scale or outlier classification is altered. This is the evidence behind the Criterion column of table 3.3: a well-formed, single-mode distribution whose spread comes from rejection sampling and scheduling noise, with only a handful of classified outliers.

## 3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap at the core boundary; the size results show where the real cost lies. Table 3.4 decomposes every LAS object into bytes at the target setting, explaining *which* component drives the size. Sizes are fixed by the encoding, so they are exact and carry no dispersion.

Two points follow. First, the response $z$ is essentially the entire signature

Table 3.4: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting, in packed wire bytes (not on-chain gas), with its notation — that of Esgin et al. [10], except that a signature carries the *digest* $\tilde{c}$, not the polynomial $c$ (table A.3). "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. Three facts read off directly: the signature, pre-signature, and adapted signature are the same serialized size (adaptation changes the *value* of the response, not its length); the response $z$ dominates every signature; and the only *public-key-shaped* object LAS adds over a basic signature is the statement $Y$ — the protocol also transmits a signature-sized pre-signature, which the table marks as added. The sizes hold for the C and the Rust implementation alike: the wire encoding is byte-identical in both by construction, asserted programmatically on every Rust run (section 3.3). Sizes at every parameter set: table A.7 (appendix).

| Object | Notation | Bytes | In the basic signature? |
|---|---|---|---|
| Public key | $pk = t$ | 4416 | yes (shared) |
| Secret key | $sk = r$ | 704 | yes (shared) |
| Challenge digest | $\tilde{c}$ | 48 | yes |
| Response | $z$ / $\hat{z}$ | 6688 | yes |
| Signature | $(\tilde{c}, z)$ | 6736 | yes (basic signature) |
| Statement | $Y = t'$ | 4416 | LAS only (public) |
| Witness | $y = r'$ | 704 | LAS only (private) |
| Pre-signature | $(\tilde{c}, \hat{z})$ | 6736 | LAS only |
| Adapted signature | $(\tilde{c}, z)$ | 6736 | LAS (verifies as basic sig) |

— 99.3% of it at every parameter set — since the challenge travels only as its short digest, so $z$ is the main target for reducing LAS signature size. Second, the adaptor layer additionally transmits a public-key-sized statement $Y$ and a signature-sized pre-signature — the only objects a basic signature does not send; the swap also carries $\pi$.

The cost of LAS is not adaptor computation but communication — specifically $z$ and $Y$. At the core tier, the adaptor operations add at most +8.2% of compute, while the objects on the wire are kilobytes. That matters more in a blockchain setting than "the signature is 6736 bytes": bytes reaching a ledger are stored and replicated by every node, while the statement and pre-signature, though off-chain, still cost both parties bandwidth. Each response coefficient is bit-packed at a fixed width, with no entropy coding — the clearest opportunity for reducing that footprint (section 5.3).

### 3.4.1 The price of post-quantum: classical adaptor baseline

The second baseline is a classical secp256k1 ECDSA adaptor signature [5]: the natural functional counterpart of LAS, driving the same scriptless swaps but resting on discrete-log rather than lattice hardness. It measures the practical cost of moving from a classical adaptor signature to a post-quantum one. LAS is instantiated at *Simplified Dilithium-II* and only there: secp256k1 offers roughly 128-bit classical security, so those dimensions are the most like-for-like engineering match, while -III or -V would pit the classical scheme against a deliberately stronger configuration. The boundaries differ between the two libraries, so table 3.5 matches them per operation and the comparison is read conservatively — the conclusions rest on order-of-magnitude gaps, not percent-level differences.

Two findings stand out. First, the post-quantum price is most pronounced in size: LAS's signature and public key are roughly 72× and 89× larger than the classical adaptor's, while every LAS operation stays well under a millisecond. Even the largest per-operation factor (Adapt, 300×) repays decomposition, because it compares two different *quantities of work* rather than two speeds for the same work: Algorithm 2 of [10] obliges Adapt to run PreVerify before returning, so 63% of LAS's 399 $\mu$s is a full lattice verification and the rest is largely codec, whereas `secp256k1_ecdsa_adaptor_decrypt` merely inverts one scalar and multiplies, leaving verification to a separate exported call [5]. Charging the classical side the verification LAS cannot skip puts the two at 102 $\mu$s against 399 $\mu$s — 4×, not 300×: a property of the two *interfaces*, not of lattice arithmetic. Second, the two schemes pay their adaptor overhead in opposite ways. The classical adaptor must attach a discrete-logarithm-equality proof [11], charging its own PreSign and PreVerify 4.5× and 4.0× their base operations, whereas LAS folds the statement into a hash it already computes. Relative to each scheme's own base operations, the lattice adaptor layer adds less computational overhead.

## 3.5 Testing the simplification: the adaptor with ML-DSA verification

Section 2.2 follows the LAS paper's simplified Dilithium-style base, which omits Dilithium's hint vector, Power2Round and high/low-bit decomposition. In this simplified form, the exact identity $A\mathbf{z} - c\,t = w + Y$ is exposed. This section tests

Table 3.5: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) against LAS at the *Simplified Dilithium-II* parameters of table 2.1. Those dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim; the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings $\mu$s/op (mean ± SD, machine of record); sizes in bytes. The two libraries draw their measurement boundaries differently, so the final column is *tier-matched* — each LAS row taken at whichever of its two tiers corresponds to the classical library's boundary for that operation. Why that matching is necessary, the boundaries themselves, the repetition counts, and the object-level differences behind the size rows are set out in appendix A.3. The two classical adaptor-layer ratios quoted in the text are each taken over *their own* base operation — PreSign over plain Sign (4.5×), PreVerify over plain Verify (4.0×) — and are *derived* from this library's single mixed native-API tier, not a paired, interleaved measurement of the kind section 3.2 reports for LAS. The corresponding LAS figures are the byte-tier ones, +15.8% for PreSign and +37.9% for PreVerify, that tier being the more conservative of the two reported for LAS.

| | ECDSA adaptor (classical, hybrid native API) | LAS adaptor (post-quantum, core tier) | LAS adaptor (post-quantum, packed tier) | overhead LAS ÷ classical (tier-matched) |
|---|---|---|---|---|
| *Computation ($\mu$s/op)* | | | | |
| KeyGen | $12.4 \pm 0.04$ | $27.9 \pm 0.2$ | $102 \pm 5$ | 2.3× |
| Sign | $17.2 \pm 0.1$ | $271 \pm 12$ | $432 \pm 17$ | 16× |
| Verify | $25.1 \pm 0.3$ | $61.9 \pm 1.0$ | $124 \pm 12$ | 2.5× |
| PreSign | $77.3 \pm 0.2$ | $292 \pm 15$ | $527 \pm 23$ | 6.8× |
| PreVerify | $101 \pm 1$ | $65.5 \pm 0.9$ | $252 \pm 6$ | 2.5× |
| Adapt | $1.3 \pm 0.01$ | $65.7 \pm 0.8$ | $399 \pm 7$ | 300× |
| Extract | $13.8 \pm 0.1$ | $24.7 \pm 1.5$ | $282 \pm 6$ | 20× |
| *Communication (bytes)* | | | | |
| Public key / statement | 33 | 2944 | | 89× |
| Secret key / witness | 32 | 512 | | 16× |
| Signature | 64 | 4640 | | 72× |
| Pre-signature | 162 | 4640 | | 29× |

whether those omitted optimisations can coexist with the adaptor layer: the four adaptor operations were built around NIST ML-DSA *as FIPS 204 specifies it* [19], all three optimisations enabled. Three findings follow; the second pinpoints the boundary.

The naive port fails outright. Adding the statement to the challenge hash and

changing nothing else — treating the hint machinery as a black box — yields pre-signatures that do not pre-verify at all (0/200), because the transmitted hint reconstructs HighBits($w$) while the challenge committed to HighBits($w + Y$). Some modification is therefore genuinely unavoidable.

But the verifier is not what must change. Once the *whole* commitment path moves onto $w+Y$ — the committed high bits, the low-bits rejection test *and* MakeHint — and PreSign tightens its bound by the witness norm, the construction satisfies the full adaptor contract (13/13 items, including four tamper rejections and byte-identical determinism), and, decisively, the unmodified FIPS 204 verifier accepts the adapted signature (200/200). So PreSign and PreVerify require adaptor-specific algorithms, while Verify does not: a post-quantum adaptor swap could settle against a standard ML-DSA verifier. A matched no-statement baseline reproduces FIPS 204's own published repetition rates, which is what makes these rows attributable to the adaptor rather than to a porting error.

The size gain is real but bounded, and it relocates the bottleneck. Measured against the scheme of record in one process, the adaptor layer stays single-digit on both constructions (+2.8% against +2.2% for PreSign per attempt), and per-signature cost is close to equal for opposite reasons: ML-DSA restarts about twice as often (5.2172 against 2.7136) but each attempt is roughly half the price. What ML-DSA buys is *bytes* — the signature falls to 3309 B from 6736 B — but the statement $Y$ does not shrink at all (byte-identical at 4416 B), because Power2Round compresses a public key while $Y$ enters the verification identity *before* any rounding. The swap payload therefore improves only to 0.69×, and $Y$ becomes larger than the signature it accompanies. Compressing the signature moves the bottleneck to the statement rather than removing it, redirecting the size question of section 3.4 and the future work of section 5.3.

This is a functional demonstration, not a security argument: whether committing to HighBits($w + Y$) preserves ML-DSA's unforgeability is not analysed, and is exactly the kind of claim section 4.4 declares out of scope.

## 3.6 UTXO swap execution result

The protocol defined in fig. 2.4 was driven end-to-end on the model UTXO ledger of section 2.7. Two results matter. First, the protocol runs and every step is asserted, including the counterfactuals that a pre-adaptation signature fails basic

verification and that $\pi$ is rejected against any other statement. Second, $\pi$ is what makes Bob's final step *guaranteed* rather than merely expected: by the Module-SIS uniqueness argument of [10] the extracted witness equals the proven ternary one, so the adapted signature is certain to clear the verification bound. Without $\pi$ a malicious counterparty could claim and leave an unadaptable pre-signature behind. The proof measures 30711–30760 B on the wire at a knowledge error at most $2^{-127}$ and is exchanged off-chain only.

This section therefore reports execution evidence, not the protocol definition itself. The UTXO cost study follows in sections 3.7 and 3.8; the separate gas-metered venue check is isolated in section 3.9.

## 3.7 The swap on a UTXO chain: three configurations compared

The execution result above establishes that the protocol runs; this section measures what it *costs*, over the UTXO ledger and under the design of section 2.7: three configurations (table 2.2), of which only the $2 \to 3$ step is controlled. Timings are the mean $\pm$ sample standard deviation over 10 complete swaps at the *packed* tier, so they are not comparable with section 3.2. Both LAS configurations passed the rejection gate (2.8510 attempts per PreSign against the closed-form 2.77483).

**The proof, not the signature, dominates execution time.** Table 3.6 gives the per-phase timings behind this comparison. The role-A proof consumes 99.3% of end-to-end time in configuration 2 and 98.6% in configuration 3. Strip it out and the entire post-quantum signature workload — two key generations, a statement, two pre-signatures, a pre-verification, two adaptations, an extraction and two settlements — costs 4468.6 $\mu$s, about 5.5$\times$ the classical swap's 819.7 $\mu$s. That increase stays millisecond-scale, while a proof of hundreds of milliseconds decides whether the protocol feels instantaneous. Even at the packed tier the signature workload is not the bottleneck — the post-quantum penalty is not where intuition puts it.

**The controlled comparison inverts the expected trade-off.** Replacing Groth16 with LaZer — same signature, same $\mathbf{A}$, same inputs — makes the swap *faster* by 53.1% while making it *larger* by 61.9%: roughly three times as quick

Table 3.6: Per-phase execution time of one complete swap, mean ± sample SD over 10 swaps ($\mu$s; AMD Ryzen 7 7745HX with Radeon Graphics). Dashes mark phases a configuration does not have. End-to-end is context, not the headline.

| Phase | Classical | LAS + Groth16 | LAS + LaZer |
|---|---|---|---|
| KeyGen ×2 + Gen | 58.4 | 540.7 | 503.2 |
| Prove ($\pi$) | — | 645621.6 | 216080.6 |
| PreSign ($u_1$) | 114.9 | 1065.3 | 1065.7 |
| ProofVerify ($\pi$) | — | 13718.1 | 90720.8 |
| PreVerify (abort gate) | 147.6 | 364.9 | 380.2 |
| PreSign ($u_2$) | 107.1 | 1070.1 | 966.7 |
| Adapt ($u_1$) | 138.4 | 392.5 | 380.5 |
| Settle chain 2 | 51.9 | 253.0 | 239.9 |
| Extract ($u_2$) | 25.3 | 185.2 | 183.6 |
| Adapt ($u_2$) | 132.7 | 353.8 | 340.9 |
| Settle chain 1 | 43.4 | 243.0 | 234.9 |
| end-to-end | 819.7 | 663808.3 | 311097.1 |
| of which $\pi$ | — | 99.3% | 98.6% |
| excluding $\pi$ | 819.7 | 4468.6 | 4295.7 |

to produce a proof, yet emitting one about 240× bigger. Groth16 is *succinct* — three group elements whatever the circuit — but producing that proof means a multi-scalar multiplication over the whole constraint system, whereas LaZer's proof grows with the statement yet proves a lattice relation with lattice machinery, avoiding the encoding of a 23-bit modulus into a 254-bit pairing-friendly field. Verification runs the other way (13718.1 $\mu$s against 90720.8), succinctness paying off exactly where it is designed to.

**Configuration 2's trade is worse than it looks.** Shor breaks Groth16's pairing-group discrete logarithms, so its proof is precisely the component a post-quantum adversary attacks: a post-quantum signature does not protect a swap whose fairness rests on a classical proof. Groth16 also needs a *trusted, per-circuit* setup (1.225142318s) whose secret must be destroyed for soundness, whereas LaZer's parameters are transparent. Configuration 3 is thus not merely the post-quantum option: it is faster end-to-end, needs no trusted party and removes a quantum vulnerability, paying in bytes and in slower proof verification.

**Communication is again the real price, with a usability consequence.** Table 3.7 gives the corresponding byte counts. A classical swap moves 941 B;

Table 3.7: Communication cost of one complete swap, in bytes, observed from the transmitted buffers over 10 swaps. Off-chain messages are counted, not only what reaches a ledger. **A range indicates a message whose size varied between runs, and only configuration 3's proof does so**: LaZer packs the proof's Gaussian response vectors with a variable-length code and then reports the length its encoder actually reached, so the packed size depends on the values sampled for that particular proof. Every other object here has a size fixed by its encoding, and the two ranged totals inherit that single varying term.

| Message | Classical | LAS + Groth16 | LAS + LaZer |
|---|---|---|---|
| *Off-chain (exchanged directly between the parties)* | | | |
| statement $Y$ | 33 | 4416 | 4416 |
| proof $\pi$ | — | 128 | 30711–30760 |
| $\hat{\sigma}_1$ | 162 | 6736 | 6736 |
| $tx_1$ | 114 | 4497 | 4497 |
| $\hat{\sigma}_2$ | 162 | 6736 | 6736 |
| $tx_2$ | 114 | 4497 | 4497 |
| **off-chain total** | **585** | **27010** | **57593–57642** |
| *On-chain (published to the ledgers)* | | | |
| $tx_1 + \sigma_1$ (chain 1) | 178 | 11233 | 11233 |
| $tx_2 + \sigma_2$ (chain 2) | 178 | 11233 | 11233 |
| **on-chain total** | **356** | **22466** | **22466** |
| **total per swap** | **941** | **49476** | **80059–80108** |

the fully post-quantum swap moves 80059–80108 B, about 85.1× more, and puts 63.1× more on the ledgers — the multiple that matters economically, since a UTXO chain prices by size. A fully post-quantum swap needs 311097.1 $\mu$s and tens of kilobytes of off-chain messaging per exchange, overwhelmingly the proof, against 819.7 $\mu$s classically: comfortable on a laptop, but a wide enough gap that a mobile wallet's assumptions would not port naively. No constrained device was measured, so that point should not be over-read.

## 3.8 Transaction structure and what a UTXO chain would charge

The sizes above are the model ledger's own encoding (appendix A.4). What a settled swap costs on a Bitcoin client is read from two *mined* transactions: an ordinary payment, and one leg of the swap of appendix A.12. Figure 3.3 compares

them structurally and table 3.8 field by field. Both were mined on regtest, the swap leg on the node carrying the experimental rule: the sizes are real, on regtest rather than mainnet.

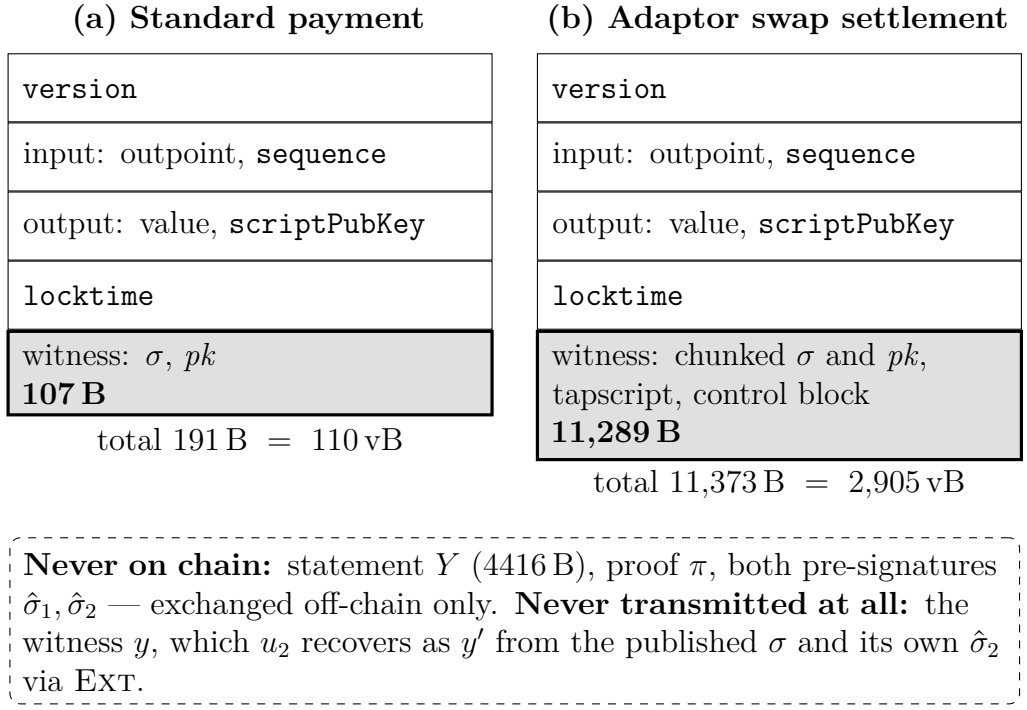



Figure 3.3: Two *mined* transactions: an ordinary payment, and one leg of the swap settled in appendix A.12. **No field is added, removed or reordered**, and the base is byte-identical at 82 B — the swap condition needs no on-chain script of its own, which is what *scriptless* means. Exactly one row differs, and the difference does not come from the adaptor layer: the witness carries lattice objects rather than an elliptic-curve signature and key, and because each stack element is capped at 520 B they arrive chunked, alongside the tapscript and its control block. Both transactions pay to the same output form, so the output does not grow; a settlement paying to taproot instead would add 12 B of base, which is not what was mined here. Sizes are the client's own, and the field layout follows [16, 14, 26]; table 3.8 gives every field.

**The witness discount absorbs more than half of the post-quantum penalty.** This rests on a transaction structure Bitcoin did not originally have. Before SegWit [16, 14] a signature travelled in the input script, where every byte was billed alike; the segregated witness bills witness bytes at one weight unit against four, and Taproot [26] supplies the script-path spend the swap input uses. Derived from the two mined transactions, the post-quantum settlement is

Table 3.8: Two mined transactions field by field, every figure as the client decoded it: an ordinary payment against one leg of the swap of appendix A.12. Total size, weight and virtual size are the client's own; the base is reconstructed from the fields and *cross-checked* against that weight, since BIP-141 fixes weight as three times base plus total [16]. "Witness stack" is the serialized items — for the swap leg the chunked signature and key, the tapscript and the control block — and "witness size" adds BIP-144's marker and flag [14]. Virtual size is the quantity a UTXO chain prices, and the gap between it and total size is the witness discount. **Scope**: both were mined on regtest, the swap leg on the node of appendix A.12 carrying the experimental rule.

| Field | Classical (B) | LAS (B) |
|---|---|---|
| *Base data — billed at 4 weight units per byte* | | |
| `version` | 4 | 4 |
| input count | 1 | 1 |
| input: outpoint + empty `scriptSig` + `sequence` | 41 | 41 |
| output count | 1 | 1 |
| output: value + `scriptPubKey` | 31 | 31 |
| `locktime` | 4 | 4 |
| **base size** | **82** | **82** |
| *Witness data — billed at 1 weight unit per byte* | | |
| `marker` + `flag` | 2 | 2 |
| witness stack (2 items / 24 items) | 107 | 11,289 |
| **witness size** | **109** | **11,291** |
| **total size** | **191** | **11,373** |
| weight (WU) | 437 | 11,619 |
| **virtual size (vB)** | **110** | **2,905** |

59.5× the classical one in raw bytes; in virtual size — what fees are charged on — only 26.4×, a post-quantum signature being *entirely* witness data. That decision, taken long before anyone considered lattice signatures on Bitcoin, removes 56 % of the size penalty; on the original structure the same settlement would pay the undiscounted rate throughout. It is a concrete argument for UTXO chains as the venue, with no counterpart on the EVM, where calldata carries no comparable discount; section 3.9 measures what a gas-metered venue charges.

**It fits the weight limits, not the element-size or verification rules.** The mined settlement weighs 11,619 WU against the 400,000 WU standardness limit — 2.9 % of it. The limit is a weight limit, not a gas limit, and one settlement transaction sits well inside it.

Two obstacles survive, neither caused by the adaptor layer. No deployed consensus rule verifies a lattice signature: Script offers `OP_CHECKSIG` over ECDSA or BIP-340 Schnorr and nothing else, so configurations 2 and 3 could not settle on Bitcoin as it stands; the application setting in [10] assumes a UTXO chain "where the signature algorithm is replaced with a lattice-based signature scheme", and this study takes no position on which consensus route would deliver it. Separately, the objects collide with size limits: the signature is 13.0× over the 520 B consensus stack-item limit and 84× over the 80 B standardness limit [4]. Both travel as length-checked 520 B chunks the new rule reassembles, satisfying the consensus limit rather than raising it. Both apply equally to an ordinary post-quantum payment: they are properties of lattice signature sizes.

**Both obstacles were put to a real client.** A stock Bitcoin Core node carries the lattice objects — refused by default relay policy, yet consensus-valid and mined — but verifies nothing: an ordinary Schnorr signature authorises it. Adding one consensus rule, an `OP_SUCCESS` opcode [27] over the byte interface of section 2.3, gives a node settling a two-leg swap with no elliptic-curve signature, 19 mutations tried being refused by it and accepted by an unmodified node — the control attributing the refusals to the new rule. The blocker is a consensus rule, not engineering. Its cost was measured: one verification runs 374 $\mu$s against the curve baseline, a BIP340 Schnorr check's 33.0 $\mu$s — 11.3× — and a signature that fails late costs about as much as one that verifies. The valid-path latency corresponds to a derived serial rate of 2.67k LAS verifications/s on this machine, not whole-node or network throughput. A patched node is not Bitcoin, so that conclusion stands; the exercise is functional, with no security argument (appendix A.12).

## 3.9 EVM verification as a separate venue check

As a separate check on deployability, native on-chain LAS verification was *implemented* and measured on a real Solidity stack: one claim transaction running full verification and releasing one leg's escrow costs ≈56.6 million gas (table 3.9; fig. 3.4), ≈748× the classical claim and over the EIP-7825 cap. The application is built on the UTXO setting assumed in [10] (section 2.7); this section measures what a gas-metered venue charges for the same verification.

That figure measures one *implementation*, not the scheme. A restructured

verifier — same parameters, bounds and challenge preimage, differing in how the computation is expressed — was mined by a real client, running full LAS verification and releasing the escrow at 16 413 275 gas: 97.8% of the EIP-7825 cap, which bounds the transaction gas limit and so covers calldata as well as execution. Its equivalence is conditional on a registration invariant, and both are set out in table 3.9. On-chain verification therefore fits in one transaction at *Simplified Dilithium-III* for the 32-byte signed message and signature instance measured, with a thin margin. Verification alone at *Simplified Dilithium-II*, charged as one transaction, is 65% of the cap; at *Simplified Dilithium-V* one transaction is exceeded, derived rather than measured (appendix A.3). The contrast with the application's own venue remains: the UTXO constraint is a transaction *weight* limit one settlement sits well inside (section 3.8).

Table 3.9: On-chain gas for the swap's claim step, measured on a local EVM (`forge -gas-report`); EVM gas is deterministic for fixed bytecode, revision, inputs and state. **These are *execution* figures** — they exclude the 21,000 intrinsic charge and the per-byte calldata a real transaction also pays under EIP-7623, which is why the under-the-cap claim in the text rests on a client receipt rather than on this table. `claimLAS` performs *no* lattice verification and is a strict lower bound; ECDSA verifies in the native `ecrecover` precompile, whereas both LAS rows run in EVM bytecode. The two `claimLASVerified*` rows decide the same predicate as `base_verify` (section 2.1), the second restructuring that computation rather than changing it — but **only under a registration invariant that neither row checks**, and where it fails a *different* predicate is decided; the equivalence testing, the invariant and the cost of breaking it are set out in appendix A.3. The client run used anvil at a pinned `osaka` revision with EIP-7825 enforced, over 50 148 B of calldata. **Scope**: *Simplified Dilithium-III*, a 32-byte signed message, that revision, and one signature instance — two stages of the verifier branch on coefficient values, so variation across instances is unquantified. The 363 941 gas of headroom is *measured*; that it corresponds to a few hundred further bytes of signed message is *derived* from it and the measured per-permutation hash cost.

| Claim path | On-chain verification | Gas | × classical |
|---|---|---|---|
| Classical ECDSA adaptor (`claimClassical`) | full, `ecrecover` precompile | 75 763 | 1× |
| LAS floor (`claimLAS`) | none (calldata + `keccak256`) | 290 640 | 3.8× |
| LAS full verify (`claimLASVerified`) | full `base_verify`, in Solidity | 56 647 414 | 748× |
| LAS full verify, restructured (`claimLASVerifiedOpt`) | full `base_verify`, in Solidity | 16 438 275 | 217× |

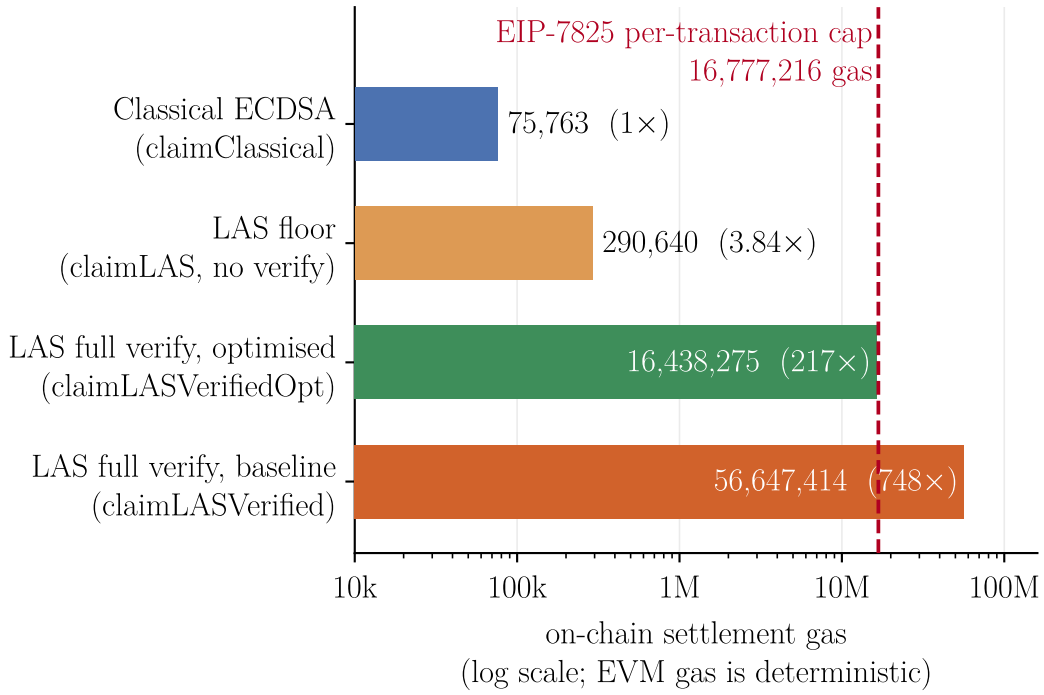



Figure 3.4: The claim paths of table 3.9 on a logarithmic axis, which is what makes the spread legible. The dashed line is the EIP-7825 cap on the transaction gas limit ($16\,777\,216 = 2^{24}$): the baseline verifier's bar crosses it, while the restructured `claimLASVerifiedOpt` — same scheme, same predicate under the registration invariant of table 3.9, different expression — falls below. All four bars come from one `-gas-report` table and so are mutually comparable; the under-the-cap claim itself rests on the separate real-client run described in table 3.9.

The same venue distinction is structural as well as numerical: fig. 3.5 shows that the EVM keeps the authorising transaction signature classical and carries the lattice verifier inputs as contract data, unlike the UTXO settlement structure of fig. 3.3.

## 3.10 Threats to validity

Several factors qualify these results. All timings are machine-dependent, mitigated by reporting standard deviations, fixing one machine and compiler, emphasising ratios over absolutes, and by the cross-implementation check (section 3.3). The parameter set's concrete security level is not formally established, security proofs being out of scope, so every cross-scheme claim is qualified by table 2.1; the

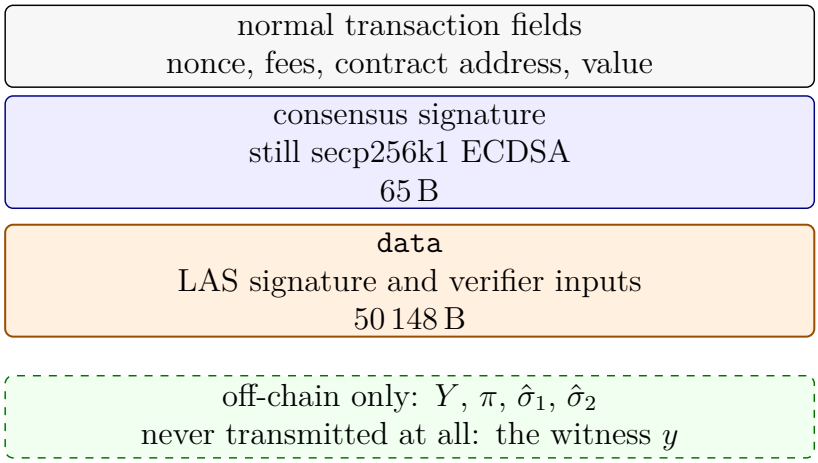



Figure 3.5: The same settlement on Ethereum: the claim transaction of the client run in table 3.9, read against the UTXO counterpart in fig. 3.3. Neither venue gains a field from the adaptor layer, but the lattice signature lands in a different structural position. On a Bitcoin-like UTXO chain it must occupy the signature slot consensus checks, so the chain needs a new consensus rule before it can verify LAS. On the EVM the authorising transaction signature remains secp256k1; the lattice signature rides in `data` as an argument to a contract. Contracts therefore make Ethereum the more flexible venue and Bitcoin the more restricted one here. The 50 148 B is measured from the client receipt and is the whole calldata payload, not the signature alone.

modulus is FIPS 204's $q \approx 2^{23}$ rather than the $2^{24}$ of [10], which leaves correctness unaffected ($q > 2\gamma$) but changes the security margin. The extracted witness is exact here, whereas the relaxed setting of [10] allows witness noise that can grow across long payment paths.

Two further factors are specific to section 3.7. The UTXO ledger is a model rather than a node (appendix A.4): real transaction objects, real serialised sizes and the real leak channel atomicity depends on, but no fees or confirmation latency, so on-chain byte counts are the honest proxy. That limit belongs to the venue rather than the study: configurations 2 and 3 need a consensus change, built and run in appendix A.12. Finally, the Stage-2 timings come from a single desktop machine (AMD Ryzen 7 7745HX with Radeon Graphics), so the usability discussion is an extrapolation, not a measurement on constrained hardware.

# Chapter 4

# Evaluation

This chapter judges the work against the objectives of section 1.3: the evaluation strategy and its alignment with those objectives, each objective against the evidence of chapter 3, then the implementation challenges that shaped the outcome and the limitations that bound it. Judgement of *what the artefact achieves* is made here; section 5.2 steps back to the process.

## 4.1 Evaluation strategy and its justification

The central evaluation question was what adaptor functionality *costs*, so each measured difference had to be attributable to one cause. Four decisions follow, each tied to an objective.

Correctness is established before any cost is reported (objective 3). A fast but wrong scheme measures nothing, and no formal proof was in scope, so correctness rests on differential evidence: a pinned known-answer value reproduced byte-for-byte by a second, independently written implementation (section 3.3). Internal self-consistency is not correctness — section 4.3 gives a case where every primitive passed in isolation and the assembled whole was still wrong.

The primary comparison is controlled, not merely convenient (objective 4). LAS is measured against its own simplified base at identical algorithm, parameters and primitives, so the difference between the two paths is the adaptor layer and nothing else. Comparing instead against the optimised CRYSTALS-Dilithium reference would confound the adaptor cost with parameter, packing and hint-vector differences; that comparison is therefore retained only as context, and the classical ECDSA adaptor only as a functionality-matched "price of post-quantum"

baseline.

Every measurement declares its boundary (objective 4). Results are reported at two tiers (section 2.6), because an undeclared boundary misleads quietly — as it did twice here before the tiers were fixed. For the same reason each run gates on a directly counted rejection rate against its closed-form prediction, rather than inferring acceptance from timing.

Metrics follow the venue (objective 5). The application study reports execution time and communication bytes with off-chain messages counted, because the evaluated UTXO ledger has no gas metric — and because off-chain messaging is where the post-quantum cost lands.

## 4.2 Achievement of the objectives

Table 4.1 sets out the evidence behind each objective. All five were met; two need more than a table row.

Objective 4's fairness claim rests on control rather than assertion: in Stage 1 the two compared paths share algorithm, parameters and primitives, and in Stage 2 the two LAS configurations share one public matrix and receive byte-identical inputs, verified at run time. The measurements are then defended from two independent directions — a closed-form gate on the rejection rate, and re-measurement under a different compiler and an independent statistical harness.

Objective 5 was met in the UTXO setting assumed in [10], with a gas-metered venue measured against it. On a local EVM a *complete*, validated native LAS verifier was measured at ≈56.6 million gas (table 3.9), above the EIP-7825 per-transaction cap. That negative result is quantified rather than assumed, and it is what the comparison contributes. That ceiling later proved to bound the *implementation* rather than the scheme: a claim transaction running a restructured verifier was mined inside the cap — under table 3.9's registration invariant, at the target set, message length and signature instance measured (section 3.9). The venue decision therefore rests on cost, not impossibility.

## 4.3 Implementation challenges

Turning the simplified-Dilithium base into LAS was conceptually small — four added operations and one extra term in a hash — but the six problems of

Table 4.1: Each objective of section 1.3 against its evidence, which spans the introduction, chapter 3 and the appendix. All five were met; the evidence column states what each rests on.

| **Objective** | **Evidence** |
|---|---|
| 1. Literature and scheme selection | Adaptor signature selected for its direct relevance to atomic swaps; LAS [10] selected as the design with the shortest path from a standardised lattice signature (section 1.2). |
| 2. Additive implementation | Lattice core reused byte-for-byte, zero upstream functions modified, made auditable by a two-branch diff; adaptor layer, wire encoding and application are new code (table A.8). |
| 3. Validation | Functional, serialization, tamper and known-answer tests pass (section 3.1); an independent Rust implementation reproduces the wire format and the pinned value byte-for-byte (section 3.3). |
| 4. Fair benchmarking | Adaptor overhead isolated per operation at two declared tiers (table 3.2); two further baselines, computation separated from communication, rejection rate gated against theory, relative conclusions reproduced by the Rust port (table 3.3). |
| 5. Atomic-swap demonstration | End-to-end two-party, two-chain swap settling both legs and asserting unspendability before adaptation (section 3.6); extended to a measured three-configuration study on a UTXO ledger (section 3.7). |

table A.1 consumed most of the engineering effort. Three themes recur. Two are failures that *pass every local test*: a loosened PreSign bound leaves the adaptor operations working while breaking ordinary verification two steps later, and a doubly-transformed public matrix produces a verifier whose parts agree with each other but not with the C implementation — both caught only by checking outside the component, which is why section 4.1 treats internal self-consistency as insufficient.

Two more are measurement artefacts that would have biased the headline result had they survived, and are why every boundary is now declared and every run gated. The third is asymptotic rather than arithmetic, and generalises past this project: in constraint-system libraries the ergonomic interface and the scalable interface are not the same interface.

## 4.4 Limitations

The validity threats of section 3.10 bound what the results *mean*. Two further limitations belong to the artefact rather than the measurement. The hint-free construction yields larger keys and signatures than optimised Dilithium — cryptographic optimisation given up for a transparent, testable artefact — and section 3.5 quantifies that price and shows it recoverable in principle, though only for the signature and not the statement. Configuration 2 instead gives up post-quantum assurance for a controlled comparison: it is the intermediate point that makes the proof-system comparison possible, not a stack anyone should deploy, since a post-quantum signature does not protect a swap whose fairness rests on a pairing assumption.

# Chapter 5

# Conclusion, critical reflection, and future work

## 5.1 Conclusions

The aim of section 1.3 was achieved within the stated prototype scope: the lattice core is reused unchanged; the adaptor layer, wire encoding and application are new tested code; functional, serialization, tamper and known-answer tests pass; Rust reproduces the pinned value byte-for-byte; and a two-party, two-chain swap runs.

The central question — what adaptor functionality costs over a plain signature, and what post-quantum operation costs over a classical one — is answered with measured data at declared boundaries. Adaptor functionality has small core-tier overhead at the target setting (+2.2%, +5.1% and +7.4% for PreSign, PreVerify and Adapt respectively), with a +8.2% core-tier maximum across the sweep, rising to +15.8–84.4% once packing is included, because the increase is codec work, not arithmetic. Communication therefore dominates the added cost: the signature is 99.3% response vector, with the statement $Y$ on top. The classical comparison points the same way, with its discrete-logarithm-equality proof making that adaptor layer proportionally more expensive.

The implementation followed the LAS paper's simplified Dilithium-style base. Rebuilding the adaptor path on ML-DSA showed that simplification is not functionally required for final verification (section 3.5). PreSign and PreVerify do require adaptor-specific algorithms — the naive port fails outright — but Verify does not: a standard FIPS 204 verifier accepts the adapted signature. That route

would halve the signature at close to equal computation, yet leave $Y$ untouched, relocating rather than removing the communication bottleneck.

Taking the protocol to the UTXO setting assumed in [10] sharpens that conclusion: at application level the signature is not the dominant cost. The role-A proof consumes 99.3% of a Groth16 swap and 98.6% of a LaZer swap. Changing only the proof system makes the fully post-quantum swap 53.1% faster end-to-end while increasing its communication by 61.9%. It needs no trusted setup and is the only configuration Shor does not break, paying in bytes and in slower proof verification.

All five objectives were met (table 4.1). The principal qualification is scope: no concrete security is formally analysed, and nothing settled on a live network.

## 5.2 Critical reflection

### 5.2.1 What was achieved

The central strategic choice — to build *additively* on a standardised reference rather than reimplement lattice arithmetic — paid off twice. It made the implementation tractable, and it produced a strong provenance argument: the security-critical arithmetic is byte-for-byte the published CRYSTALS-Dilithium reference, with a two-branch diff that makes the claim auditable. Being language-agnostic, the strategy also made the second implementation cheap, moving the correctness argument from one tested codebase to two agreeing byte-for-byte and exposing timing conclusions to an independent harness. Pairing a pinned known-answer value with an independent reimplementation proved a decisive way to validate a cryptographic port.

Two further things went better than expected. Comparing LAS against a parameter-matched base rather than optimised Dilithium turned an apparent size deficit into an attributed one. Counting rejection attempts rather than inferring them from timing also corrected a materially wrong earlier figure, now a gate every run must pass.

Two negative results also matter. Compressing the statement inside the construction and trying the relation under a succinct post-quantum system were implemented rather than left as speculation, and neither paid off (section 5.3). Each now names the mechanism that defeats it, closing the first direction outright

and the second at the scale measured here.

### 5.2.2 What fell short

Three shortfalls are worth stating plainly, over and above the scoped limitations of section 4.4.

First, on-chain settlement was measured, then solved narrowly, and still not deployed. The first Solidity verifier sat far above the cap. The dissertation then drew the wrong lesson, treating the gap as structural because SHAKE256 is not EVM-native. Re-expressing the same computation closed it (section 3.9). What remains unsolved is cost: settlement is far more expensive than a classical claim, and the thin margin holds for one signature instance and message length.

Second, the applications are exploratory demonstrators, not products. Stage 2 measures over ledger models; the real-client runs are local, not live-network deployments. For the UTXO study this is partly forced: Bitcoin Script cannot verify a lattice signature, so the post-quantum configurations need a consensus change to realise the setting assumed in [10]. The reported costs are implementation-level feasibility estimates, not deployment figures or upper bounds.

Third, concrete security of the chosen parameter set is unanalysed by design. This sanctioned scope decision bounds what the performance numbers mean: they characterise a faithful, testable artefact, not a parameter set anyone should deploy.

### 5.2.3 What would be done differently

The clearest change is target selection for the application. Too much effort went into Solidity before the gas cost was known. The framing should have been where a post-quantum swap actually spends, not whether one signature can be verified on chain. Starting from the UTXO setting assumed in [10] would have reached section 3.7 sooner, with the EVM retained as a gas-metered venue check.

Second, measurement discipline would come first: the biased acceptance estimate and asymmetric timed loops both followed from building the benchmark before fixing its boundary. Third, the ML-DSA experiment should have come *before* the simplified scheme was justified in writing: testing it took days, not weeks.

## 5.3 Future work

- **A statement that is small by design.** With the signature halved and $Y$ untouched (section 3.5), the statement is the dominant object. Compressing $Y$ *inside* this construction was tested and fails: truncation is invisible to the adaptor's own functions, which hash whatever statement they are handed, and fatal at both boundaries that matter: ordinary verification recomputes $A\mathbf{z} - c\,t$ against the true $Y$, and EXTRACT accepts only on the exact relation. Deriving $Y$ from its seed hands over the witness. What is open is a different hard relation whose statement is small by construction.

- **Reduce the proof.** The route proposed here was measured and fails at this scale. In the swap, $\pi$ must prove knowledge of a short witness to $Y$ [10]; hiding that witness is required by the swap, while succinctness was this dissertation's added goal. Measured under LaBRADOR, that encoding loses to the deployed prover on size — by the library's own estimate rather than a packed proof — and on time outright: succinctness is asymptotic, and one instance is too small to reach it. What remains open is a proof small at *this* scale.

- **Reduce the cost of one-transaction verification.** Fitting under the cap was proposed here as future work and then answered, though only for one signature instance and message length at both measured sets — verification alone at the smaller — with the largest exceeded by derivation (section 3.9). Fitting is not the same as being worth doing: settlement remains far above a classical claim, and the hash is the dominant measured term. A precompile for ML-DSA verification is specified as a draft, covering only NIST level II [8], and was declined from the next scheduled upgrade [3]; it would not verify LAS, bearing instead on the ML-DSA route of section 3.5, whose adapted signatures a stock FIPS 204 verifier accepts.

- **Analyse the security of the ML-DSA variant.** Section 3.5 establishes that committing to HighBits$(w + Y)$ is *functionally* sound; whether it preserves ML-DSA's unforgeability argument is unexamined, and is the prerequisite before that route could be recommended over the simplified scheme.

- **Take the UTXO study to a live network.** *Signet* would supply the

classical configuration with the fees, propagation and confirmation latency the model cannot. For configurations 2 and 3 no stock Bitcoin test network can supply a deployment figure: the blocker is consensus, not engineering. Appendix A.12 adds that rule experimentally and settles a lattice-authorised two-leg swap; Bitcoin as deployed cannot, so that figure awaits adoption.

# Appendix A

# Supporting evidence and reproducibility

This appendix carries the supporting implementation, methodology and reproducibility material that would interrupt the main report if presented in full: the commands needed to reproduce the benchmarks and tests, the derivations deferred from chapter 2, the representative PreSign, Adapt and Extract routines, supplementary benchmark data, and the real-client blockchain experiments. The complete source and evidence artefacts remain in the project repository.

## A.1 Building and reproducing the benchmarks

The benchmarks and tests are reproducible from the commands below; the application experiments — the Stage-2 swap, the on-chain measurements and the real-client node runs — are recorded with their own evidence paths in the later sections of this appendix. This report's numbers are regenerated from the C and the Rust logs together, so on a clean checkout the C suite saves its evidence and defers the report sync, leaving the report untouched; running the Rust suite next completes it. The upstream Dilithium reference is pinned at the repository's initial commit (`2374d22`); the classical baseline library is the BlockstreamResearch `secp256k1-zkp` `ecdsa_adaptor` module at commit `95b9835`; the proof-of-knowledge library is LaZer at commit `2fa3dfb`; the Rust port builds on a vendored copy of the `fips204` crate (commit `c948882`). The two external libraries are unmodified, and the commands below explicitly check out the pinned `secp256k1-zkp` and LaZer commits rather than their current default branches.

The C toolchain is the system compiler (`cc`, Ubuntu GCC 13.3.0) with `-O3`; the Rust toolchain is stable `rustc` (the committed run used 1.96.0) in release profile.

Listing A.1: Fair benchmark and tests (first-stage evaluation). The suite script saves the Stage-1 C logs and parsed tables as a timestamped evidence run; the application targets listed individually below are outside that subset. It then calls the report sync, which regenerates this report's figures, tables and inline numbers from the C and the Rust logs together — so on a clean checkout, where the Rust logs do not yet exist, that step defers and leaves the report untouched until the suite in listing A.4 has run.

```
# the supported evidence pipeline: Stage-1 tests + benchmarks, then
    report sync.
# STAGE1_ONLY=1 skips the four application targets -- test_contract3,
    test_pcn3 and
# bench_app3, superseded pre-seven-type files that no longer compile,
    and test_swap3,
# which is current but needs the LaZer build below.
# This is how the Stage-1 run of record (evidence/latest) was produced.
STAGE1_ONLY=1 scripts/run_benchmark_suite.sh

# or individually:
cd ref
# fair, parameter-matched base-vs-adaptor benchmark at each parameter
    set
make test/bench_levels_paper && ./test/bench_levels_paper
make test/bench_levels2 && ./test/bench_levels2
make test/bench_levels3 && ./test/bench_levels3 # target setting
make test/bench_levels5 && ./test/bench_levels5
# correctness, serialization/tamper, known-answer tests
# (test_serde_l3 is the TARGET set (6,5,49); test_serde3 builds the
    paper dims)
make test/test_las3 && ./test/test_las3
make test/test_serde_l3 && ./test/test_serde_l3
make test/test_kat3 && ./test/test_kat3
# end-to-end atomic swap incl. the proof of knowledge pi
# (needs the one-time LaZer build below)
make test/test_zkp3 && ./test/test_zkp3
make test/test_swap3 && ./test/test_swap3
```

Listing A.2: Classical adaptor baseline (one-time clone).

```
git clone https://github.com/BlockstreamResearch/secp256k1-zkp
    third_party/secp256k1-zkp
git -C third_party/secp256k1-zkp checkout 95b9835 # the measured commit
cd ref && make test/bench_classical && ./test/bench_classical
```

Listing A.3: Proof-of-knowledge library (one-time clone and build; see the repository README for the dependency recipe).

```
git clone https://github.com/lazer-crypto/lazer third_party/lazer
git -C third_party/lazer checkout 2fa3dfb # the measured commit
cd third_party/lazer && make lazer.h liblazer.a
# the generated proof parameters (ref/relation_zk_params.h) are
    committed,
# so SageMath is needed only to REgenerate them, never to build
```

The Rust port is reproduced with the standard Cargo tooling; the test gate runs first because the benchmarks refuse to time unverified code, and the known-answer test asserts the same pinned value as the C implementation.

Listing A.4: Rust port: test gate, statistical benchmark, protocol driver, and size report (one suite script, or the four steps individually).

```
# the one supported Rust evidence pipeline (tests + benchmarks + report
     sync)
scripts/run_rust_bench_suite.sh

# or individually:
cd rust/fips204-las
# conversion::tests cover the vendored crate's hint/Power2Round helpers,
     which the
# LAS port does not use (it has its own serialize.rs), so the gate
    skips them
cargo test --lib --tests -- --skip conversion::tests # gate: known-
    answer value
cargo bench --bench las_bench # Criterion.rs statistical harness
cargo run --release --example bench_levels # protocol driver (mirrors
    the C driver)
```

```
cargo run --release --example size_report # packed component sizes
```

## A.2 The timing-benchmark procedure

The pseudocode below is the procedure both protocol drivers implement (`ref/test/bench_levels.c` and the Rust `examples/bench_levels.rs`, deliberately line-for-line mirrors), backing the methodology of section 2.6. Ordering matters: correctness is asserted *before* timing on the very objects about to be timed, so no failure path is ever measured, and the rejection gate runs over the *timed* calls themselves, so the gate certifies the same data the tables report. The same procedure runs once per parameter set.

Listing A.5: The timing-benchmark procedure (both tiers). $R = 5$ repetitions; $N = 500$ (sign-class) or 1000 (verify-class) iterations; the rejection gate follows eq. (2.2).

```
Input: parameter set (n, l, kappa); fixed pp seed; fixed 33-byte
   message
 1 Setup: expand A from the fixed seed; KeyGen; relation Gen (Y, y)
 2 Gate 0 (correctness): run the full adaptor contract on the exact
   objects about to be timed (PreVerify accepts; basic Verify rejects
   the pre-signature; Adapt verifies; Extract returns y exactly);
   abort the run on any failure
 3 for each operation op in {KeyGen, Sign, Verify, PreSign, PreVerify,
   Adapt, Extract}: # core tier
 4 for r in 1..R:
 5 a0 <- attempt counter # sign-class only
 6 t0 <- monotonic clock
 7 repeat N times: run op (structures in / structures out;
            fresh masking randomness inside every sign-class call)
 8 run_r <- (clock - t0) / N
 9 att_r <- (attempt counter - a0) # sign-class only
10 report mean +/- sample SD over run_1..run_R
11 Gate 1 (run validity): for Sign and for PreSign, the mean attempts
   per call over ALL R*N timed calls must satisfy
       |mean - E| <= 5 * E*sqrt(1-1/E) / sqrt(R*N)
   where E = 1/p_acc is the closed-form expectation; abort on failure
12 repeat steps 3-11 at the packed boundary (packed bytes in /
```

```
   packed bytes out, decode + encode inside the call)
13 emit packed sizes (communication is fixed: measured once, no SD)
```

The Criterion.rs harness measures the same operations under its own protocol (3 s warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals) and applies the same Gate 1; it shares no timing code with the drivers, which is what makes its agreement an independent check rather than a re-run.

## A.3 Methodology details

This section carries the derivations and experimental-design detail deferred from chapter 2, so that the body states the decisions and this appendix justifies them in full.

**Why the per-attempt acceptance probability is secret-independent.** Each coefficient of the mask is uniform on the $2\gamma + 1$ values of $[-\gamma, \gamma]$, and the secret-dependent term satisfies $\|c\,r\|_\infty \le \kappa$, because the challenge has exactly $\kappa$ nonzero coefficients of $\pm 1$ and the secret is ternary. Each response coefficient $z_i = y_i + (c\,r)_i$ is therefore uniform on a window of $2\gamma + 1$ consecutive integers whose centre is displaced from zero by at most $\kappa$. Sign's acceptance interval $[-(\gamma - \kappa),\, \gamma - \kappa]$ is narrower than that window by exactly $\kappa$ on each side, so it lies wholly inside the window *wherever* the displacement falls. Each coefficient is thus accepted with probability exactly $\big(2(\gamma - \kappa) + 1\big)/(2\gamma + 1)$, independently of the secret — which is precisely why rejection sampling makes the published response simulatable, and hence why it leaks nothing about $r$. Raising this to the $(n + \ell)\,d$ coefficients of one attempt gives eq. (2.2). PreSign repeats the derivation at its own, tighter bound $\gamma - \kappa - 1$: that is why the two expectations differ slightly, and why the gate below is applied to each separately.

**The run-validity gate in full.** Over a run's timed sign-class calls the measured mean attempts $\bar{A}$ must satisfy $|\bar{A} - E| \le 5\,\sigma/\sqrt{n_{\text{calls}}}$, where $E = 1/p_{\text{acc}}$ is the closed-form expectation of eq. (2.2) and $\sigma = E\sqrt{1 - 1/E}$ is the geometric standard deviation. The test is applied separately for Sign and for PreSign, at both timing tiers, in C and in Rust, and a run that fails it aborts instead of producing evidence. The resulting band is $\pm\sim 8\%$ at the C drivers' 2500 timed calls and $\pm\sim 1\%$ at the Criterion harness's $\approx 10^5$, which is why only the latter resolves the 2% theoretical

separation between Sign and PreSign. The gate is a consistency check on the observed rejection rate, not a proof of sampler correctness: it provides evidence that the rejection behaviour is consistent with theory and that rejection-sampling variation is sufficiently controlled for the reported mean.

**The Groth16 circuit.** No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and it is far cheaper than the statement suggests. The map $r' \mapsto Ar'$ is linear with public coefficients, so each row is an affine combination of witness variables and needs no witness–witness multiplication — the expensive ingredient in a rank-one constraint system. The public linear map is still enforced, through linear constraints; the only non-linear and range constraints are the ternary bound, enforced as $r'^3 = r'$ per coefficient, and the reduction modulo $q$, enforced with a range-checked quotient per row. The constraint count is therefore linear in the parameter set's dimensions rather than in their product, and the same statement would have been intractable had it involved products of two *secret* polynomials. Because the matrix is held in the number-theoretic-transform domain and is exported by neither implementation, it is recovered by evaluating the scheme's own linear map on unit vectors, so the circuit encodes the map the signature scheme actually computes with rather than a re-derivation that could drift from it.

**Measurement invariants of the UTXO study.** Two invariants are enforced rather than assumed. A phase must be timed in *every* run or in none, so that no mean is silently taken over the subset of swaps that reached it; and communication subtotals are computed within each run before being reduced across runs, since summing each message's minimum would describe a swap that never occurred. One-time costs — expanding $A$, building the elliptic-curve context, and Groth16's trusted setup — are performed before timing begins and reported separately, being amortised over every swap rather than paid per swap. The rejection gate applies here too, but on a dedicated batch of 2000 pre-signature calls rather than on the two per swap the protocol itself performs: over so few calls the five-sigma band is wider than the distance from the expected attempt rate to a loop that never rejects at all, so a gate on that sample would have been vacuous.

**Matching the boundaries of the classical comparison.** The two libraries of table 3.5 do not draw their measurement boundaries in the same place, so the

comparison states both. The classical column is the library's *hybrid* native-API boundary: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call (packed-like), whereas KeyGen, Sign and Verify exchange in-memory structs with their wire codecs outside it (core-like). LAS is therefore shown at both its core and its packed tier, measured on the C implementation, and the tier-matched column pairs each operation with whichever LAS tier corresponds to the classical boundary for that operation — the core tier for KeyGen/Sign/Verify and the packed tier for the four adaptor operations. Timings are the mean ± sample standard deviation over 10 runs × 1000 iterations classically and 5 repetitions × 500/1000 iterations for LAS, on the same machine. Comparing every row against a single LAS tier instead would count the wire codec on one side only, for four of the seven operations, which is why the matching is done per operation rather than per table. Two object-level differences lie behind the size rows: LAS KeyGen also produces the statement and witness, which take the public-key and secret-key size, and the classical pre-signature is a syntactically distinct object — an ECDSA signature plus a discrete-logarithm-equality proof — whereas the LAS pre-signature shares the signature format exactly.

**What the restructured on-chain verifier changes, and how it was checked.** Subject to the registration invariant set out in the next paragraph, which neither verifier checks, the two `claimLASVerified*` rows of table 3.9 decide the same predicate as `base_verify`: decode, norm bound, $w' = A\mathbf{z} - c\,t$, and the SHAKE256 challenge check. The first mirrors those steps directly from the vendored ZKNox primitives and is validated against the C reference. The second re-expresses them: the public key is registered in the number-theoretic-transform domain and $c\,t$ folded before a single inverse transform, parameters are read from calldata rather than decoded into memory, and the sponge avoids the per-byte absorb. It reproduces the first verifier's packed $w'$ byte-for-byte on the C golden vectors, and agrees with it on accept and reject for the golden signature and for a tampered response, challenge and message.

**The registration invariant, and what is *not* enforced.** Neither verifier derives its public parameters: the matrix arrives already transformed in both, and the restructured one additionally takes the public key twice — transformed for the arithmetic, packed for the challenge preimage $\mathrm{pack}(t) \,\|\, \mathrm{pack}(w') \,\|\, M$, which is the only one of the three the hash ever sees. Equality with `base_verify`

therefore holds exactly where three things are true together: the registered matrix is $\mathrm{NTT}(A')$ for the agreed $A'$, the registered key is $\mathrm{NTT}(t)$ for some key $t$, and the packed bytes that are hashed are $\mathrm{pack}(t)$ for that *same* $t$. The escrow contract commits to all three at funding and re-derives that commitment at claim time, which stops a *claimer* substituting; it establishes nothing about their mutual consistency, and no on-chain step recomputes one from another. The obligation rests on whoever funds the escrow.

What follows from breaking it is, in general, only that a *different* predicate is decided — not that a weaker one is, and no analysis here covers the general case. Some registrations are weaker, though, and one suffices to show the condition is load-bearing rather than pedantic: register both transformed objects as zero and every product vanishes, leaving $w' = \mathbf{z}_{\mathrm{top}}$ and an acceptance test that anyone can satisfy for any in-bound $\mathbf{z}$, with no key and no signature. The escrow would then pay out without the adapted signature ever reaching a chain — and that publication is exactly what leaks the witness and makes the swap atomic. Custody is unaffected, since the beneficiary is fixed at funding and the claim entrypoint pays that address whoever calls it; what a bad registration costs its own funder is the atomicity the escrow was bought for. Recomputing $\mathrm{NTT}(t)$ from the packed bytes once at funding, off the verification path, would discharge the two conjuncts about $t$; the one about $A'$ would need the untransformed matrix, or the seed it expands from, supplied at registration as well. Neither is implemented, neither is costed here, and the measured figures are for the verification path as it stands.

**Carrying the one-transaction result to the neighbouring parameter sets.** The restructured verifier fixes its dimensions at build time, so a second copy was instantiated at *Simplified Dilithium-II*: duplicated rather than parameterised, so that nothing could move the Dilithium-III figures already measured, and given its own golden vectors exported from the C implementation. What is charged there is *verification only* — the calldata goes to a harness entry point that runs the verifier and returns its verdict, so none of the escrow work in the claim path is included: no context re-derivation, no state transition, no event, no payout. Charged as one transaction under EIP-7623 it comes to 10 956 784 gas, 65% of the cap, 5 820 432 short of it, with the standard branch rather than the calldata floor binding (`evidence/onchain_d2/`). It is therefore a harness computation over a narrower boundary than the Dilithium-III client receipt of section 3.9,

and the two are not interchangeable. The accepted verdict is asserted before the figure is reported — a rejecting run would have measured the early exit rather than verification — though the assertion follows the measurement rather than preceding it.

*Simplified Dilithium-V* was not built, and the asymmetry between the two directions is why a derivation could still settle it. Deriving that a set *fits*, rather than measuring an instance of it, needs an upper bound over the whole computation, and two stages resist one: the challenge sampler's rejection loop and the response decoder both branch on coefficient values. Deriving that a set is *exceeded* needs only a lower bound, which tolerates dropping exactly those stages. The derivation therefore pushes the calldata the way that favours fitting, subtracts both data-dependent stages whole, counts every remaining stage at zero growth — each being a loop over the dimensions with no value-dependent branch — and adds one measured quantity: the difference between the two challenge-preimage absorptions, taken against a fixed-size memory arena so that the scratch allocated inside the timed call cancels rather than inflating the difference. That this measured difference clears the gap the other terms leave is what the result rests on (gate `evm/test/LASShakeGrowth.t.sol`, arithmetic `scripts/derive_onchain_d5_bound.py`, run `evidence/onchain_d5bound/`). What follows is that one transaction is exceeded at that set; no gas total for it follows, a lower bound not being a value, and neither does any claim about what further optimisation would achieve there.

The Dilithium-II and Dilithium-III figures are each one signature instance, for the same reason those two stages resist an upper bound, and variation across instances is unquantified at every set: nothing here supports calling it negligible, and nothing here supports calling it material. The Dilithium-V statement is not an instance at all, but a bound assembled from Dilithium-III measurements and one measured absorption difference.

**One deliberate exception to reproducibility.** Every other input derives from the pinned seed, but Groth16's trusted setup and each of its proofs draw fresh operating-system entropy. Both are security requirements rather than preferences: a reproducible setup would leave the toxic waste recoverable and void soundness, and reusing proof randomness across statements breaks zero-knowledge. The measurement consequence is stated rather than hidden — proof *size* is unaffected,

since a Groth16 proof is three group elements regardless, but proving *time* includes the entropy draw and so carries a little extra run-to-run variance.

## A.4 The modelled UTXO ledger

The ledger behind section 2.7 is a model, not a node: it has no peer-to-peer layer, no mempool policy, no fee market, no reorganisations and no script interpreter. What it does provide is what the study needs — real transaction objects with canonical serialisation, a signature hash computed over the transaction rather than over an abstract message, and the public observability on which the protocol's atomicity depends, since a settled transaction's signature can be read back off the ledger by anyone, which is exactly how the witness leaks. An output carries an amount plus a spending condition: a signature check under a public key, with an optional timelocked refund branch.

One boundary belongs with the atomicity claim. In the measured swap each coin is funded to an output that its own holder's key controls, and no conflicting spend of a funded outpoint is modelled. What the runs establish is therefore that the honest ordering settles both legs and that the witness leaks as the protocol requires — not that a party who has already claimed is unable to race to respend its own funded outpoint. Figure 1 in [10] does not specify a funding or locking step, so adversarial funding races lie outside both the figure and this experiment.

Deploying instead on a real Bitcoin node is not a matter of effort. Bitcoin Script cannot verify a lattice signature, so configurations 2 and 3 could not settle there without a consensus change — the setting in [10] instead assumes a UTXO-based chain *whose signature algorithm is replaced with a lattice-based signature scheme*, which is why that sentence is stated as an assumption rather than left implicit.

## A.5 Wire encoding and the typed codec

The packed field widths behind section 2.3 are 23 bits per coefficient for the public key and statement, 2 bits for the ternary secret and witness, and 18 or 19 bits for the response $z$ depending on the parameter set, since that field must hold $2(\gamma - \kappa)$ and $\gamma$ grows with $\kappa\, d\, (n + \ell)$. Validation is deliberately *asymmetric*, and the asymmetry is worth stating precisely because it decides where a bad object is

caught. Decoding rejects a public-key or statement coefficient $\geq q$ and the invalid ternary code, but a signature decodes *permissively*: $\tilde{c}$ is copied raw and $z$ is read with the FIPS 204 BitUnpack of [19]. The response band $[0, 2(\gamma - \kappa)]$ does not fill the fixed-width field that carries it, so some byte strings decode to a $z$ outside the valid band. That is intentional: there is no decode-time norm test to fail. The norm rejection belongs to Verify, which recomputes the challenge from the decoded response and compares digests. Encoding is the strict direction: it refuses a non-ternary secret or witness and a response exceeding $\gamma - \kappa$, so the serializer cannot emit an out-of-band response, even though one can be handed to the decoder. The byte-level entry point `base_verify_packed` decides the predicate over these bytes, and it — not the decoder — is what rejects all 6736 tampered signatures in section 3.1.

One encoding decision is worth recording. Following FIPS 204 [19], the wire format carries not the challenge polynomial $c$ but only the digest $\tilde{c}$ from which it is derived, verification re-deriving $c = \mathsf{SampleInBall}(\tilde{c})$ after decoding. This is a computation-versus-storage trade: storing the expanded polynomial would save that re-derivation on every verification, while storing the digest keeps the signature smaller.

The digest *width* is an implementation choice rather than a requirement of the LAS construction [10], which specifies $H$ only as a random oracle onto the challenge set $C$ and fixes no encoding. It is therefore taken from FIPS 204, whose SampleInBall consumes a seed of $\lambda/4$ bytes, with $\lambda = 128, 192, 256$ for ML-DSA-44, -65 and -87 giving 32, 48 and 64 bytes. The width tracks the LAS parameter set and not the reference build mode, since the sweep compiles several LAS sets against a mode chosen only to satisfy the reference's parameter header: the ML-DSA-65-aligned target set takes 48 bytes, the ML-DSA-44- and ML-DSA-87-aligned LAS sets take 32 and 64, and the historical paper-parameter reference set, which corresponds to no ML-DSA parameter set, takes 32 by this project's choice. Matching these reusable parameters aligns the primitives and the challenge-digest strength only — LAS retains its own ternary secret distribution, rejection bounds, exact hint-free relation and adaptor algorithms — so it is not a claim that the scheme is ML-DSA-65 or inherits its security category.

## A.6 The atomic-swap demonstration and its proof of knowledge

The protocol defined in fig. 2.4 follows Figure 1 in [10] step for step, realised over the model ledger of appendix A.4. Two parties swap coins across two ledgers with no trusted third party and no swap-specific on-chain script — ordinary output scripts remain, which is what *scriptless* means here. Alice, the witness holder, commits first: she draws a fresh statement/witness pair $(Y, y)$, produces a zero-knowledge proof $\pi$ that $Y$ has a *ternary* preimage $(\exists\, r' : \mathbf{A}r' = Y \wedge \|r'\|_\infty \leq 1)$, pre-signs the transaction spending her own coin, and sends $\{Y, \pi, \hat{\sigma}_1\}$ in one off-chain message. Bob pre-signs his own transaction against the *same* $Y$ only after both $\pi$ and Alice's pre-signature verify — the protocol's abort gate; at that point neither pre-signature is spendable. Alice, who knows $y$, adapts and publishes her signature to claim Bob's coin; Bob, holding the matching pre-signature, extracts $y$ from that published signature and uses it to adapt and publish his own. In the measured executions both legs settle and the witness is recovered as the protocol requires; the adversarial case is outside what this demonstration establishes (appendix A.4). The demonstration prints this narrative and asserts every step, including the counterfactuals that a pre-adaptation signature fails basic verification and that $\pi$ is rejected against any other statement.

The proof $\pi$ reuses the LaZer lattice zero-knowledge library [15] rather than a proof system written from scratch — the same reuse posture applied throughout. Because LaZer proves per-partition binary coefficients or exact Euclidean norms, the ternary statement is encoded by binary decomposition, $[\mathbf{A} \mid -\mathbf{A}]\,(r_+ \,\|\, r_-) = Y$ with both halves proven binary, which is equivalent to knowledge of a ternary preimage $r' = r_+ - r_-$. The generated parameter set reports a knowledge error at most $2^{-127}$ under Module-SIS and a proof size of 32 046 B — the parameter set's own figure, larger than the proof bytes observed on the wire in table 3.7 and not to be conflated with them — with proving and verifying each completing well under a second. A dedicated test suite asserts completeness, rejection of every sampled single-byte tamper, rejection against a wrong statement, and that the prover refuses a non-ternary witness. The modelled ledger of appendix A.4 additionally tests a separate timeout/refund edge case that Figure 1 in [10] omits. In this one-chain test, a coin is funded to an output claimable by the counterparty and refundable by its funder after a timeout; the counterparty never responds, an

early refund is rejected as a *timing* failure rather than a cryptographic one, and after the timeout the funder recovers its coin.

## A.7 The six implementation challenges

The themes drawn from these six challenges are discussed in section 4.3; the table itself is placed here because it runs to more than a page.

Table A.1: The six challenges that shaped the implementation. A seventh, cross-cutting difficulty — reproducing the C scheme byte-for-byte in Rust, down to how each language's uniform sampler discards rejected blocks — is discussed with the cross-validation results (section 3.3); the pinned known-answer value is what turned it into a mechanical process. One caveat belongs with the gas figure of challenge 5: the reused ZKNox primitives come from a self-described non-production library, so the measurement is the honest cost of *a* numerically-correct verifier, not an endorsement of that library.

| Challenge | The problem | Resolution and lesson |
|---|---|---|
| 1. The rejection-bound budget | PreSign must reject at $\gamma - \kappa - 1$, one tighter than eq. (2.1). At $\gamma - \kappa$ instead, PreSign, PreVerify and Extract all still succeed, but the *adapted* signature can exceed the bound and be rejected by ordinary Verify — silently, probabilistically, two operations from the cause. | Prevented by reasoning through the norm budget (an honest witness has $\|r'\|_\infty \leq 1$, hence $\|\hat{\mathbf{z}} + r'\|_\infty \leq \gamma - \kappa$) rather than found by testing. |
| 2. Reference optimisations appear to fight the adaptor identity | Power2Round, the hint vector and high/low-bit decomposition discard low-order information, which appeared to rule out the exact identity $A\mathbf{z} - c\,t = w + Y$; disabling them was treated as a design requirement. | Establishing *which* to disable was a prerequisite to a working PreVerify. The belief that they *had* to be was later tested and proved too strong: with the whole commitment path moved onto $w + Y$ the adaptor works with all three enabled (section 3.5): a sufficient route, not a necessary one. |

*continued on the next page*

*Table A.1 — continued from the previous page*

| Challenge | The problem | Resolution and lesson |
|---|---|---|
| 3. Living in another scheme's namespace | The reference globally defines single-letter parameter macros — its `N` is the ring degree, its `K` the module rank — while LAS needs different dimensions with the same natural names, so a constant silently inherits the wrong value through the preprocessor instead of failing to compile. | Forced prefixed parameters and a documented C–Rust–paper name map. |
| 4. Benchmarking a variable-time scheme fairly | Rejection sampling makes sign-class timings heavy-tailed. An acceptance rate inferred from a timing ratio was biased ($\approx$23% where direct counting gives 37.1%), and the two languages' base signing paths initially used different norm-check strategies *inside* the timed loop — invisible to the known-answer test, but a work-profile asymmetry biasing the comparison. | Attempts are now counted and gated against theory, the drivers mirror each other line-for-line, and every boundary is declared (section 2.6). |
| 5. An error that was self-consistent | Porting the verifier to Solidity, the public matrix $A'$ — stored by the reference *already in the NTT domain* — was transformed a second time. The recomputed commitment and challenge agreed *with each other*, so every primitive passed in isolation while the whole disagreed with the C implementation. | Found only by recomputing the challenge digest against the reference's; thereafter the port was validated in stages against exported vectors. |
| 6. A constraint system that was asymptotically wrong | The Groth16 circuit first accumulated its dense rows with ordinary field-element arithmetic, cloning the accumulated linear combination on each addition and so costing work quadratic in the row width. The symptom was an apparent hang, the work happening during setup before the driver prints. | Rebuilding each row once as an explicit linear combination against the low-level constraint-system interface made the cost linear; setup now completes in 1.225142318s. |

## A.8 Test types and parameter notation

This section collects the testing taxonomy (table A.2), the parameter notation (table A.3) and the measurement boundaries (table A.4) used throughout the evaluation.

Table A.2: The four test types, what each asserts, and what it would catch. The functional suite's statement-binding clause — that a pre-signature *fails* basic verify — is the tripwire that distinguishes an adaptor signature from a signature; the known-answer value is what the second implementation is checked against (section 2.5).

| Test type | Scale and assertions | Catches |
|---|---|---|
| Functional | 1000 iterations per Dilithium mode (2, 3, 5), fresh parameters, keys and message each time; asserts the full adaptor cycle — pre-signature pre-verifies, *fails* basic verify, adapts to a valid signature, and extraction returns the witness exactly — plus a sign/verify round trip and one-bit-forgery rejection | any break in the adaptor contract; statement binding |
| Serialization | 256 random instances; exact round-trip for keys and every signature variant | encoder/decoder asymmetry |
| Tamper | every one of the 6736 bytes of a valid packed signature flipped in turn; all must break verification; malformed public-key and secret-key encodings rejected at decode | a verifier that accepts tampered bytes; a key decoder that trusts its input |
| Known-answer | all inputs fixed, deterministic API, packed bytes of the public key, secret key, ordinary signature, pre-signature and adapted signature folded into one SHAKE256 digest compared against a pinned 32-byte value | any unintended change to sampling, algebra or encoding, on any machine |

Interpreting those tests across the parameter sets requires a common notation. Table A.3 therefore collects the symbols used throughout the construction and the value each takes at the four evaluated settings; the figures and tables in chapter 3 refer back to these definitions rather than restating them. The notation follows the LAS paper [10]: the ring degree is its $d$, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. Two entries are exceptions, the challenge digest $\tilde{c}$ and its width $|\tilde{c}|$: the LAS construction [10] specifies $H$ only as a random oracle onto the challenge set $C$ and fixes no encoding, so both the digest and its width are implementation choices, taken from FIPS 204 wherever a set aligns with an ML-DSA one (appendix A.5).

Table A.3: Security and scheme parameters: the symbol, its meaning, and the value(s) it takes across the parameter sets, *listed in the row order of* table 2.1 (LAS-2020/845 reference; Simplified Dilithium-II; -III; -V). The notation follows the LAS paper [10]: the ring degree is its $d$, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. The exceptions are the challenge digest $\tilde{c}$ and its width $|\tilde{c}|$: the LAS construction [10] specifies $H$ only as a random oracle onto the challenge set $C$ and fixes no encoding, so both the digest and its width are implementation choices, taken from FIPS 204 wherever a set aligns with an ML-DSA one (appendix A.5). The figure and table captions in chapter 3 refer back to these names and values.

| Symbol | Meaning | Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V |
|---|---|---|
| $n$ | module rank: rows of $A$, length of public key $t$ | 4, 4, 6, 8 |
| $\ell$ | extra columns of $A'$ (secret-vector extension) | 4, 4, 5, 7 |
| $M = n + \ell$ | response dimension (number of polynomials in $z, \hat{z}, y$) | 8, 8, 11, 15 |
| $\kappa$ | challenge Hamming weight (number of $\pm 1$ in $c$; Dilithium $\tau$) | 60, 39, 49, 60 |
| $\gamma$ | rejection / masking bound, $\gamma = \kappa\, d\, (n + \ell)$ | 122 880, 79 872, 137 984, 230 400 |
| $d$ | ring degree $(X^d + 1)$ | 256 (all sets) |
| $q$ | modulus from the reused NTT, $q \approx 2^{23}$ | 8 380 417 (all sets) |
| $c$ | challenge polynomial, the output of $H$: $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Expanded from $\tilde{c}$ by the signing and verifying paths after decoding, never transmitted | element of $R_q$ (all sets) |
| $\tilde{c}$ | challenge *digest*: the SHAKE256 output this implementation derives the challenge from, following FIPS 204's pattern $c = \mathsf{SampleInBall}(\tilde{c})$. This, not $c$, is what a signature carries | width below (all sets) |
| $\|\tilde{c}\|$ | challenge-digest width in bytes: FIPS 204's $\lambda/4$ at the three ML-DSA-aligned sets; a project choice at the LAS-2020/845 reference set, which that rule does not cover | 32 (project choice), 32, 48, 64 |

Notation fixes the symbols and parameter values used to describe each operation; it does not fix what a timing includes. Table A.4 closes that gap with the three measurement boundaries used in this dissertation. Comparisons are only ever made within one of them, which is why the classical baseline is recorded as a hybrid rather than folded into a single tier.

Table A.4: The measurement boundaries. Comparisons are only ever made within one boundary. The classical column is not a uniform tier but a *hybrid*, because `secp256k1-zkp` exposes the pre-signature only in packed form: the four adaptor operations behave like the packed tier and the three base operations like the core tier.

| Boundary | What crosses it | What it answers |
|---|---|---|
| Core tier | in-memory structures in, structures out | what the adaptor *algebra* costs, isolated from encoding |
| Packed tier | packed wire bytes in and out, including the decode of every input and encode of every output (section 2.3) | what a wire or on-chain consumer pays per call with nothing cached; one operation's byte boundary, *not* a whole-protocol cost |
| Hybrid native API (classical only) | the boundary `secp256k1-zkp` itself exposes: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call; KeyGen, Sign and Verify exchange parsed structs with the wire codecs outside it | the classical cost as the unmodified library actually charges it |

## A.9 Supporting benchmark evidence

The tables in chapter 3 carry the primary results; this section gives the supporting computation data produced by the commands above (listing A.1).

### A.9.1 Supplementary computation and rejection-sampling evidence

Table 3.2 and fig. 3.1 are the primary computation result in the main text. The supporting exhibits here give the parameter-sensitivity and rejection-sampling evidence behind the same conclusion: fig. A.1 shows that the core-tier adaptor overhead remains below +8.2% across the full sweep, while table A.5 and fig. A.2 compare the measured rejection behaviour with the geometric model and the validity gate.

Table A.5: Rejection-sampling statistics at the target setting *Simplified Dilithium-III*: the closed-form geometric model of eq. (2.2) against every measured sample. "Mean" is attempts per call and "Accept." is the per-attempt acceptance rate — the two quantities every source records, and the ones the model predicts. The per-call *dispersion* is recorded in the source CSV rather than repeated here. The C driver is the implementation of record; the Rust protocol driver and the Criterion harness are the independent samples of section 2.5. Every measured row passes the five-standard-error validity gate of section 2.6.

| Operation | Source (calls) | Mean | Accept. |
|---|---|---|---|
| Sign | geometric model, eq. (2.2) | 2.719 | 36.8% |
| | C driver, distribution sample (2000) | 2.696 | 37.1% |
| | C driver, timed-run gate (2500) | 2.711 | 36.9% |
| | Rust driver, timed-run gate (2500) | 2.704 | 37.0% |
| | Rust Criterion, gate (94395) | 2.726 | 36.7% |
| PreSign | geometric model, eq. (2.2) | 2.775 | 36.0% |
| | C driver, distribution sample (2000) | 2.804 | 35.7% |
| | C driver, timed-run gate (2500) | 2.746 | 36.4% |
| | Rust driver, timed-run gate (2500) | 2.766 | 36.1% |
| | Rust Criterion, gate (94395) | 2.778 | 36.0% |

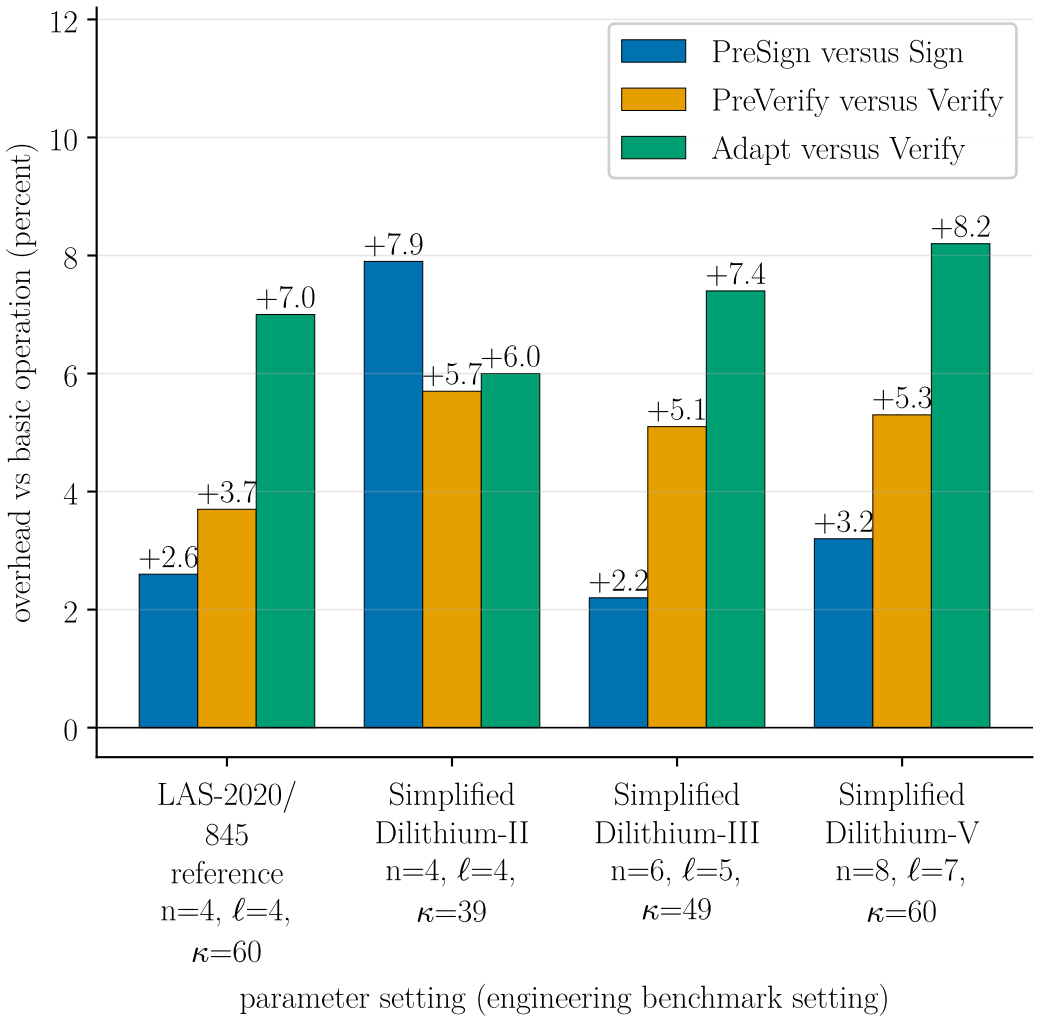



Figure A.1: Parameter sensitivity of the adaptor overhead: each LAS adaptor operation as a percentage over the basic operation it mirrors, at the four parameter sets of table 2.1. PreSign is paired with Sign; PreVerify and Adapt with Verify; Extract has no basic analogue and is omitted. Core tier only, and the premium remains small across the sweep: all target-setting adaptor operations are single-digit, and the maximum over all parameter sets is +8.2%. This is a secondary robustness check on the headline of fig. 3.1, not a result about scaling: the primary comparison remains the per-operation overhead at the target setting, table 3.2. The absolute timings behind every bar are in table A.6.

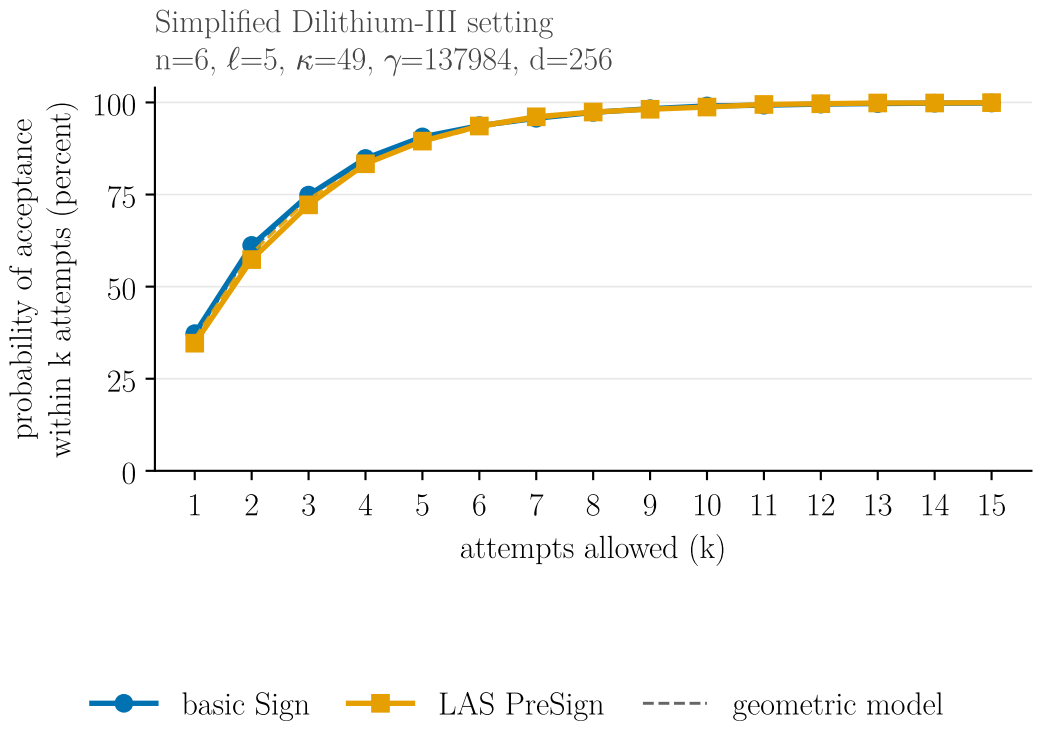



Figure A.2: The probability that a sign-class call has been accepted *within* $k$ rejection-sampling attempts, at the target setting *Simplified Dilithium-III*. **The solid curves are measured**: the empirical distribution of the same 2000-call sample table A.5 summarises, counted per attempt on the C implementation — basic Sign (blue) under the bound $\gamma - \kappa$, LAS PreSign (orange) under the tighter $\gamma - \kappa - 1$. The dashed lines are the closed-form geometric model of eq. (2.2) for those two bounds, predicting first-attempt acceptance of 36.8% and 36.0% respectively. The two measured curves are nearly coincident, which is exactly why the PreSign-versus-Sign timing gap is so small. Both pass 90% within six attempts — five clears it for Sign but not for PreSign, whose bound is the tighter of the two — and then flatten: allowing many more attempts buys almost nothing.

### A.9.2 Complete timing and component tables

Table A.6 lists every core-tier per-operation timing with its standard deviation (the data plotted in fig. 3.1, and the source of the core block of table 3.2); table A.7 gives the component byte sizes at every parameter set (the target setting is the body table table 3.4).

Table A.6: Per-operation timing, reported independently ($\mu$s per operation; mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500 sign-class / 1000 verify-class iterations; `bench_levels`, machine of record in section 2.6). KeyGen, Sign, and Verify *are* the basic signature and are reused unchanged by LAS; PreSign, PreVerify, Adapt, and Extract are the four adaptor operations. Public-parameter Setup, a one-time global cost, runs in 22–79 $\mu$s and is omitted. Parameter sets and symbols: tables 2.1 and A.3.

| Operation | LAS-2020/845 reference $(4, 4, 60)$ | Simplified Dilithium-II $(4, 4, 39)$ | Simplified Dilithium-III $(6, 5, 49)$ | Simplified Dilithium-V $(8, 7, 60)$ |
|---|---|---|---|---|
| *Basic signature (reused unchanged by LAS)* | | | | |
| KeyGen | $30.2 \pm 0.04$ | $27.9 \pm 0.2$ | $44.4 \pm 1.4$ | $67.7 \pm 0.7$ |
| Sign | $235 \pm 5$ | $271 \pm 12$ | $403 \pm 17$ | $492 \pm 11$ |
| Verify | $63.8 \pm 0.4$ | $61.9 \pm 1.0$ | $91.6 \pm 0.5$ | $134 \pm 4$ |
| *LAS adaptor operations* | | | | |
| PreSign | $242 \pm 7$ | $292 \pm 15$ | $412 \pm 10$ | $508 \pm 20$ |
| PreVerify | $66.2 \pm 0.8$ | $65.5 \pm 0.9$ | $96.2 \pm 1.6$ | $141 \pm 5$ |
| Adapt | $68.3 \pm 0.5$ | $65.7 \pm 0.8$ | $98.3 \pm 0.9$ | $145 \pm 3$ |
| Extract | $25.4 \pm 0.1$ | $24.7 \pm 1.5$ | $35.9 \pm 0.5$ | $58.0 \pm 0.2$ |

Table A.7: LAS objects by component (packed bytes; `bench_levels`). The LAS-2020/845 reference set shares the Simplified Dilithium-II dimensions. The challenge travels only as its digest $\tilde{c}$, whose width this implementation takes from FIPS 204's $\lambda/4$ rule at each aligned set (table A.3); the response $z$ grows with $n + \ell$ and dominates the signature at every setting. The target-setting column is the body table table 3.4.

| Component | Simplified Dilithium-II $(4, 4)$ | Simplified Dilithium-III $(6, 5)$ | Simplified Dilithium-V $(8, 7)$ |
|---|---|---|---|
| $\tilde{c}$ (challenge digest) | 32 | 48 | 64 |
| $z$ (response) | 4608 | 6688 | 9120 |
| Signature $(\tilde{c}, z)$ | 4640 | 6736 | 9184 |
| $z$ as % of signature | 99.3% | 99.3% | 99.3% |
| Public key $pk$ | 2944 | 4416 | 5888 |
| Secret key $sk$ | 512 | 704 | 960 |
| Statement $Y$ (LAS only) | 2944 | 4416 | 5888 |
| Witness $y$ (LAS only) | 512 | 704 | 960 |
| Pre-signature (LAS only) | 4640 | 6736 | 9184 |

## A.10 Auditing the reuse claim

Table A.8 gives the per-file classification behind the "reused vs. modified vs. added" claim of section 2.2. The claim was established by keeping the pristine upstream tree on its own branch and diffing it against the project branch: the commands below print nothing for the lattice core, which is what makes "zero upstream functions modified" an auditable statement rather than an assertion.

Table A.8: Source files of the implementation, classified by how the reference is used. The lattice core is reused unchanged; inside the reference tree the only modified files are `ref/Makefile`, which adds targets, and `ref/test/.gitignore`, which lists the new test binaries. The branch diff also shows one deletion, `ref/sign copy.c` — an annotated copy of `ref/sign.c` carried in the pinned baseline, differing from it by comment blocks only, named by no build rule, and removed here. The adaptor scheme, serialization, application integration, and evaluation harnesses are project-specific additions.

| Category | Files | Role |
|---|---|---|
| Reused unchanged | `poly/polyvec/ntt/reduce/rounding`, `packing`, `sign`, `fips202`, `params` | the entire lattice core |
| Modified | `Makefile`, `test/.gitignore` | new targets; built artefacts |
| Added | `las.{c,h}`, `serialize.{c,h}`, `relation_zk.{c,h}` (+ LaZer bridge), `basesig`, `setup`, `relation`, tests, benchmarks | this work |

Listing A.6: Branch diff establishing that the lattice core is untouched (no output means identical).

```
git diff --name-status dilithium-baseline main -- ref/
git diff dilithium-baseline main -- ref/poly.c ref/ntt.c ref/sign.c \
    ref/packing.c ref/polyvec.c ref/reduce.c ref/rounding.c ref/params.h
```

## A.11 Selected code listings: PreSign, Adapt and Extract

Three routines carry the adaptor mechanism. PreSign folds the statement into the commitment before hashing and rejects at the tighter bound; Adapt adds the witness to the pre-signature's response; Extract recovers the witness by subtraction and confirms it against the statement. They compose the same reference-derived polynomial arithmetic used throughout the base path, with the LAS-specific response-vector subtraction required by Extract. The latter recomputes $A\,s$ using the same matrix and vector operations that key generation uses to form $t$.

Listing A.7: Selected implementation cores from `ref/las.c`. The PreSign excerpt retains the adaptor-specific statement binding, challenge construction, response computation and rejection gate, while eliding the reference-derived matrix/vector setup around them; `las_adapt` and `las_ext` are shown in full, apart from abridged provenance comments.

```
/* --- PreSign: the adaptor-specific core, Algorithm 2 steps 2-7 (
    abridged) --- */
rej:
  ++las_attempts;
  las_polyvecm_uniform_Sgamma(y, mask_seed, mask_nonce++); /* y <-
     S_gamma^(n+l) */

  /* ... w = A y, via the reference-derived matrix and vector helpers
     ... */
  las_polyvecn_caddq(w);

  /* THE core adaptor mechanism: the statement is folded into the
     commitment
   * before hashing. Sign hashes w; PreSign hashes w + Y, so the adapted
   * signature satisfies the ORDINARY Verify equation for the same
      challenge. */
  las_polyvecn_add(w_plus_t_prime, w, Y->t_prime);
  las_polyvecn_reduce(w_plus_t_prime);
  las_polyvecn_caddq(w_plus_t_prime);
  las_polyvecn_pack_w(w_packed, w_plus_t_prime);

  shake256_init(&state);
  shake256_absorb(&state, t_packed, sizeof t_packed);
  shake256_absorb(&state, w_packed, sizeof w_packed); /* the shifted w +
      Y */
  shake256_absorb(&state, m, mlen);
  shake256_finalize(&state);
  shake256_squeeze(c_tilde, LAS_CTILDEBYTES, &state);
  las_poly_challenge(&c, c_tilde); /* SampleInBall(c~) */
  c_hat = c;
  poly_ntt(&c_hat);
```

```
  las_polyvecm_pointwise_poly_montgomery(presig->z_hat, &c_hat, r_hat);
  las_polyvecm_invntt_tomont(presig->z_hat);
  las_polyvecm_add(presig->z_hat, presig->z_hat, y); /* z_hat = y + c r
      */
  las_polyvecm_reduce(presig->z_hat);
  if(las_polyvecm_chknorm(presig->z_hat, bound)) /* caller-supplied */
    goto rej; /* PreSign bound */
  memcpy(presig->c_tilde, c_tilde, LAS_CTILDEBYTES); /* sigma_hat = (c~,
      z_hat) */

/* --- Adapt --- */
int las_adapt(signature *sig, const pre_signature *presig,
              const uint8_t *m, size_t mlen, const statement *Y,
              const witness *r_prime, const public_key *pk,
              const public_params *pp) {
  if(las_preverify(presig, m, mlen, Y, pk, pp))
    return -1;
  memcpy(sig->c_tilde, presig->c_tilde, LAS_CTILDEBYTES); /* challenge
     preserved */
  las_polyvecm_add(sig->z, presig->z_hat, r_prime->value); /* z = z_hat
      + r' */
  las_polyvecm_reduce(sig->z);
  return 0;
}

/* --- Extract --- */
int las_ext(witness *s, const signature *sig, const pre_signature *
    presig,
            const statement *Y, const public_params *pp) {
  unsigned int i, j;
  poly s_1_hat[ELL], a_s[LAS_N];
  las_polyvecm_sub(s->value, sig->z, presig->z_hat); /* s = z - z_hat */

  las_polyvecm_reduce(s->value);
  for(j = 0; j < ELL; ++j) /* a_s = A s, formed */
    s_1_hat[j] = s->value[LAS_N + j]; /* as KeyGen forms t */
  las_polyvecl_ntt(s_1_hat);
  las_polyvec_matrix_pointwise_montgomery(a_s, pp->a_prime, s_1_hat);
```

```
  las_polyvecn_reduce(a_s);
  las_polyvecn_invntt_tomont(a_s);
  las_polyvecn_add(a_s, a_s, s->value);
  las_polyvecn_reduce(a_s);
  las_polyvecn_caddq(a_s);
  for(i = 0; i < LAS_N; ++i) /* accept iff A s == Y */
    for(j = 0; j < LAS_D; ++j)
      if(a_s[i].coeffs[j] != Y->t_prime[i].coeffs[j])
        return -1;
  return 0;
}
```

## A.12 Settling on a real node

The transaction sizes of section 3.7 are arithmetic over the serialisation of [16, 14, 26]. This appendix records what happened when transactions of those shapes were put to a real Bitcoin client, in two stages that must not be conflated: the first carries lattice-sized objects past a *stock* node, the second verifies a lattice signature on a *patched* one. Full method, scope and caveats are in `docs/03-results/TWO_LEG_REAL_CLIENT_EXPERIMENT.md`; the evidence is under `evidence/btc_regtest/` and `evidence/btc_las_node/`.

**Carriage on stock Bitcoin Core.** Against an unmodified Bitcoin Core v31.1 regtest node (source commit `9be056a8a72b`), three transactions were constructed with the client's own test framework, offered to two nodes differing only in relay policy, broadcast and mined.

| transaction | vsize (vB) | weight (WU) | default relay policy |
|---|---|---|---|
| 1-in/1-out P2WPKH, signed by the node | 110 | 437 | accepted |
| 1-in/1-out P2TR key path | 111 | 444 | accepted |
| carriage of a LAS signature and key | 2,939 | 11,753 | `bad-witness-nonstandard` |

Three observations. The two reference spends reproduced the published figures the size model self-checks against, so that model is now one a client has confirmed

rather than one this study asserts. The carriage transaction is refused by default relay policy — the reason string is the node's own — yet is consensus-valid, was mined, and the default-policy node accepted the containing block: standardness and validity are distinct questions, and only the second bears on whether a post-quantum settlement is possible. Finally, because a signature of 6,736 bytes and a key of 4,416 bytes each exceed the 520-byte consensus limit on a witness stack item, they travel as 13 and 9 chunks respectively, which is why this transaction carries 25 witness items where an ordinary payment carries two.

Nothing here verifies a lattice signature. Stock Script has no such opcode; the spend is authorised by an ordinary Schnorr signature and the lattice bytes are discarded from the stack.

**Verification on a patched node.** [27] reserves the `OP_SUCCESS` opcodes as an upgrade path. Taking one of them (0xbb) and defining it as a lattice verification opcode over the byte interface of section 2.3, with the signed message being the transaction digest of [26], gives a node that can settle a spend carrying no elliptic-curve signature at all. Such a spend was accepted and mined (2,917 vB, 11,667 WU, 24 witness items); the consensus diff is recorded as a patch against the same release (`sha256:6fb8161d5b2b`) and applies to a pristine checkout of it.

That a node accepts a transaction proves little on its own, so each case was put to both the patched node and a stock one. On the stock node the opcode is still `OP_SUCCESS`, so it accepts every variant unconditionally — which is what makes it a control, since it cannot be reacting to the signature. 9 mutations were tried: altering the payment's value or destination, spending a different coin, computing the digest against a false input value, presenting a signature made for another transaction or over something that is not a transaction digest at all, truncating or transposing a chunk, and substituting a key the output never committed to. Every one was refused by the patched node and accepted by the stock node, which attributes the refusals to the new rule rather than to some other defect.

**A whole swap, not a single spend.** Verifying one signature is not settling a swap. The property the protocol exists for is that the party who does *not* hold the witness — here $u_2$, since $u_1$ generates $(Y, y)$ and holds $y$ throughout — obtains it only from what the counterparty published. That was put to the patched client too, across two regtest chains sharing no coins and diverging from their first block. Each leg's signed message is its own transaction digest [26], so it commits to that

leg's input, value and recipient; the two digests differ, which is what stops one signature settling both. PreSign and PreVerify run on both legs, $u_1$ adapts leg B and settles it, and the adapted signature is then read back out of the *mined* witness — its boundary located by the key hash the funding output committed to rather than by an assumed signature length. Leg A is adapted with the recovered witness and settles on the other chain. Each leg is 2,905 vB, 11,619 WU and 24 witness items: 82 bytes of non-witness transaction, the two bytes BIP-141 adds as a marker and flag [16], and 11,289 bytes of witness. That witness holds the signature and the public key split across stack elements of at most 520 bytes, followed by the tapscript and its control block.

Two separate facts carry the provenance, and they should not be merged. The recovered signature is 6,736 bytes and byte-identical to what Adapt produced, which establishes that the chain carried exactly that signature and that the recovery read it faithfully. What attributes leg A's *witness* to the ledger is different: Extract runs as a separate program, given only the recovered bytes as an argument and unable to reach the copy the adapting party held. 19 negative controls were refused by the patched node and accepted by the stock one, among them the cross-leg case — leg B's settlement signature presented on leg A. Evidence: `evidence/btc_twoleg/`.

One asymmetry with the Ethereum settlement of table 3.9 is worth stating, since it is a property of the venue rather than of the scheme. Bitcoin binds the transaction, not the chain: the digest of [26] carries no chain identifier, whereas the Ethereum escrow's message is derived from `block.chainid` among other state. Offering leg B's settled transaction to the other chain is therefore refused because that coin does not exist there — which evidences two separate ledgers, and not a signature bound to a chain.

**One swap across two kinds of ledger.** Both legs above settle on Bitcoin. A further experiment put one leg on the patched client and the other on Ethereum, so that a single LAS instance — one public matrix and one adaptor statement, with one signing key pair per leg — is verified by two mechanisms that share no code: the consensus opcode above, and the deployed escrow contract of section 3.9, which receives the matrix as calldata. The Bitcoin leg settles first; its adapted signature is recovered from the *mined* witness, Extract runs on those bytes as a separate program, and the recovered witness adapts the Ethereum leg, which

settles inside one transaction under the EIP-7825 cap. What this adds to the two-leg run above is not the choreography, which is the same, but that the two halves are different ledger models: the venue changes while the cryptographic object does not.

The timeout side was exercised on both venues, each on a recovery-test object of that venue's own kind — a control coin on Bitcoin, a second escrow on Ethereum — rather than by refunding either settled leg, since both were claimed. Each was refused before its own deadline and, once matured, returned the coin. The two deadlines are set as absolute UNIX-second values so their ordering is checkable, with the leg settled first carrying the shorter one, following the classical setup recalled in Section 4.1 of [10] — the LAS choreography of Figure 1 restates no timeout, so this is an addition around that figure rather than part of it. The ordering is *configured and checked, not enforced*: Bitcoin judges a timestamp locktime against median time past and the contract against `block.timestamp`, so the two are one numeric domain and not one clock, and neither venue inspects the other's deadline. None of this establishes a fair atomic swap. The Bitcoin claim leaf is a single-key check under the funder's own key, so the funder need not wait for the timeout, and the mempool exposes the adapted signature before confirmation, leaving a conflicting-spend race that a timeout does not close. Evidence: `evidence/btc_eth_swap/`.

Three limits travel with this. A patched node is not Bitcoin, so the claim that configurations 2 and 3 cannot settle on Bitcoin as deployed is unaffected. Implementing one of the routes sketched in section 3.7 demonstrates feasibility and takes no position on which route should be adopted. And the exercise is functional: no security argument is offered for the new rule.

**What the rule charges a validating node.** Changing a consensus rule buys something and charges something, and the charge falls on every node that ever validates the chain, so the cost was measured rather than left as a remark. The quantity that matters is script-validation time per input, since that is what scales with the number of signatures in a block. Timed over 10 repetitions × 200 iterations, paired and interleaved, one `OP_CHECKLASSIGVERIFY` costs $374 \pm 9\,\mu$s against $33.0 \pm 3.3\,\mu$s for a BIP340 Schnorr check and $31.7\,\mu$s for ECDSA — 11.3× and 11.8× respectively. Both sides start from *serialized* bytes and parse inside the timed call, as the interpreter does per input; timing a packed lattice verifier

against an already-parsed curve key would charge the wire codec to one side only. The reject path is a *valid* signature checked against a different 32-byte digest, so every stage before the final challenge comparison must still run — a corrupted signature can trip an early structural check, which would measure how quickly the verifier gives up rather than what a rejection costs. Rejection comes to 372 $\mu$s, 1.00× an acceptance. The narrow reading is the only one supported: a LAS signature that fails at the final comparison costs a node approximately as much as one that verifies, so there is no early exit to be had on this path. It says nothing about the rule's denial-of-service surface more broadly — malformed inputs take other paths, which were not timed. Runner `scripts/run_btc_las_bench.sh`; evidence under `evidence/btc_las_bench/`.

Three caveats bound those figures, and they are separate claims. First, the curve baseline is a pinned `libsecp256k1-zkp` — a fork of `libsecp256k1` running the same verification algorithms, but not the library the patched client links, so these are not Bitcoin Core's own numbers. Second, the security of the consensus modification remains unanalysed: a timing figure is not a safety argument, and soundness, denial-of-service surface and upgrade path are all untouched. Third, the two schemes are not at a matched security level — `secp256k1` offers roughly 128-bit *classical* security, whose engineering match is Simplified Dilithium-II, while the node compiles the rule at Simplified Dilithium-III because that is what it actually runs. The ratios therefore measure the curve against a deliberately stronger lattice setting and so *overstate* what the rule costs relative to a level-matched pairing; stating the direction of that bias is the point, and neither pairing is a security-equivalence claim. Finally, this is a per-input cryptographic cost only: block download, UTXO lookup, script parsing and signature caching are excluded, and the unmodified client pays those too.

# Bibliography


[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 635–664. Springer, 2021.

[2] Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations. Cryptology ePrint Archive, Paper 2026/625, 2026. `https://eprint.iacr.org/2026/625`.

[3] Tim Beiko. EIP-7773: Hardfork meta — glamsterdam. Ethereum Improvement Proposals, no. 7773, 2024. Records which proposals are scheduled for, or declined from, the Glamsterdam upgrade. `https://eips.ethereum.org/EIPS/eip-7773`.

[4] Bitcoin Core developers. Transaction relay and mining policy limits (`src/policy/policy.h`). Source code, 2026. `https://github.com/bitcoin/bitcoin/blob/master/src/policy/policy.h`; `MAX_STANDARD_TX_WEIGHT`, `MAX_STANDARD_P2WSH_STACK_ITEM_SIZE`.

[5] Blockstream Research. secp256k1-zkp: Ecdsa adaptor signature module. Software library, 2024. `https://github.com/BlockstreamResearch/secp256k1-zkp`, commit 95b9835.

[6] Maxime Buser, Rafael Dowsley, Muhammed Esgin, Clémentine Gritti, Shabnam Kasra Kermanshahi, Veronika Kuchta, Jason Legrow, Joseph Liu,

Raphaël Phan, Amin Sakzad, et al. A survey on exotic signatures for post-quantum blockchain: Challenges and research directions. *ACM Computing Surveys*, 55(12):1–32, 2023.

[7] CRYSTALS team. CRYSTALS-Dilithium reference implementation. Software repository, 2023. `https://github.com/pq-crystals/dilithium`, commit 2374d22.

[8] Renaud Dubois and Simon Masson. EIP-8051: Precompile for ML-DSA signature verification. Ethereum Improvement Proposals, no. 8051, October 2025. Draft. `https://eips.ethereum.org/EIPS/eip-8051`.

[9] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR transactions on cryptographic hardware and embedded systems*, pages 238–268, 2018.

[10] Muhammed F Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In *European Symposium on Research in Computer Security*, pages 378–397. Springer, 2020.

[11] Lloyd Fournier. One-time verifiably encrypted signatures a.k.a. adaptor signatures. Manuscript, 2019. `https://github.com/LLFourn/one-time-VES`.

[12] Brook Heisler, Jorge Aparicio, and contributors. Criterion.rs: statistics-driven micro-benchmarking for Rust. Software repository, 2025. `https://github.com/criterion-rs/criterion.rs`, version 0.8.2.

[13] Ruslan Kysil, István András Seres, Péter Kutas, and Nándor Kelecsényi. poqeth: Efficient, post-quantum signature verification on ethereum. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 327–343, 2025.

[14] Johnson Lau and Pieter Wuille. BIP 144: Segregated witness (peer services). Bitcoin Improvement Proposal, 2015. `https://github.com/bitcoin/bips/blob/master/bip-0144.mediawiki`; defines the marker/flag and witness serialisation format.

[15] Lazer-crypto project. LaZer: a library for lattice-based zero-knowledge proofs. Software library, 2025. `https://github.com/lazer-crypto/lazer`;

accompanies the CCS 2024 paper "The LaZer Library: Lattice-Based Zero Knowledge and Succinct Proofs for Quantum-Safe Privacy".

[16] Eric Lombrozo, Johnson Lau, and Pieter Wuille. BIP 141: Segregated witness (consensus layer). Bitcoin Improvement Proposal, 2015. `https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki`; defines the witness field, transaction weight and virtual size.

[17] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[18] Dustin Moody, Ray Perlner, Andrew Regenscheid, Angela Robinson, and David Cooper. Transition to post-quantum cryptography standards. NIST Internal Report 8547, initial public draft, November 2024. `https://csrc.nist.gov/pubs/ir/8547/ipd`.

[19] National Institute of Standards, Technology (NIST), Thinh Dang, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, and Angela Robinson. Module-lattice-based digital signature standard, August 2024.

[20] Andrew Poelstra. Scriptless scripts. Presentation, Milan Bitcoin meetup / Blockstream, 2017. `https://github.com/apoelstra/scriptless-scripts`.

[21] Sebastien Riou, Jong-Yeon Park, Liga Anwar, Axel Poschmann, and Michael Hutter. Apples, oranges, and signatures: Pitfalls and methodology in ML-DSA benchmarking. Cryptology ePrint Archive, Paper 2026/1333, 2026. `https://eprint.iacr.org/2026/1333`.

[22] Eric Schorn. fips204: FIPS 204 ML-DSA in pure Rust. Software repository, 2025. `https://github.com/integritychain/fips204`, vendored at commit c948882.

[23] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.

[24] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. Post-quantum adaptor signature for privacy-preserving off-chain payments. In *Financial Cryptography and Data Security*. Springer, 2021.

[25] Luke Valenta and Kevin Guthrie. Post-quantum authentication to origins is now supported. Cloudflare Blog, July 2026. Published 29 July 2026. `https://blog.cloudflare.com/post-quantum-authentication-to-origins/`.

[26] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 341: Taproot: SegWit version 1 spending rules. Bitcoin Improvement Proposal, 2020. `https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki`.

[27] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 342: Validation of taproot scripts. Bitcoin Improvement Proposal, 2020. `https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki`.